Soft-Decision Viterbi Decoder Design and Verification

Project Author

Abstract

Viterbi Decoder is a maximum-likelihood decoding algorithm used for convolutional codes, that is, it is used to select, among all possible encoded sequences, the one most likely to have produced the current received signal as the decoding result [1], [2]. When convolutional codes transmit data, the current input bit and several previous historical bits participate in encoding together, so what the receiver sees is not isolated bits, but a sequence of encoded results with “historical memory”.

The role of Viterbi Decoder is to find, among all possible historical paths, the path most likely to have produced the current received sequence. It usually expands all candidate paths into a trellis, which is a graph structure formed by unfolding all possible state transitions of a convolutional code over time, used to systematically represent all candidate paths; then at each step it calculates the branch metric based on the received data, which is the local cost measuring the matching degree between a state transition at one step and the received data, and then accumulates these local costs into a path metric, which is the total cost accumulated from the starting point to the current state, used to evaluate the quality of an entire path, and finally through ACS（Add-Compare-Select）, which is the core operation that accumulates metrics, compares values, and keeps the optimal candidate path, it continuously retains the best candidate path under each state, and uses traceback, that is, tracing backward from the final optimal state along the recorded path, to recover the original input bit sequence.

This project implements a soft-decision Viterbi Decoder intended for verification on the ZU4EV development board. Soft-decision means that decoding does not only use 0/1 decision results, but also uses the confidence of the received signal, such as multi-bit quantized values. The receiver sees continuous values with noise. Soft-decision uses multiple bits to represent the reliability degree of the received signal, rather than rigidly merging it into only 0 or 1, enabling Viterbi decoding to distinguish candidate paths more accurately in noisy environments, thereby significantly reducing the bit error rate

In terms of input form, this project supports two types of decoding input: hard-decision and soft-decision. Hard-decision only represents the received result as 0 or 1, so the decoder can only calculate distance based on whether bits are the same. Soft-decision preserves more received confidence information. For example, the 3-bit soft symbol in this project uses 0 to 7 to indicate whether a bit is more like 0 or more like 1, so the decoder can use “how certain it is” to make finer path selections.

In the ideal case, if there is no noise, the decoding problem is relatively simple, and Viterbi Decoder can easily recover the original data. This kind of test can only verify whether the function is basically correct, but it cannot reflect the performance of the algorithm in a real communication environment. In practical communication systems, signals are inevitably affected by various random interferences during transmission, such as thermal noise and circuit noise. These noises can usually be well approximated by a Gaussian distribution, so AWGN has become the most classical and most commonly used channel model

In this project, AWGN（Additive White Gaussian Noise）, a classical channel model that superimposes random noise following a Gaussian distribution onto the signal, is introduced. The purpose is to artificially superimpose random noise with controllable strength onto the transmitted signal, thereby constructing input data closer to real scenarios. In this way, what the receiver obtains is no longer ideal 0/1, but continuous values with uncertainty, which is exactly the premise for soft-decision to take effect. Through this method, it is possible to verify whether the decoder can still correctly select paths and correct errors under noisy conditions, and to evaluate its noise resistance; in addition, using AWGN can also systematically adjust the signal-to-noise ratio（SNR）, and observe changes in decoding performance under different noise intensities, such as the increase or decrease of the bit error rate

The design goal is to complete a full hardware closed loop from the algorithm model to real board-level operation. First, a Python model is used to establish a comparable golden reference, then unified test vectors are generated, then the RTL decoding core is implemented, and synthesis, implementation, and resource/timing analysis are completed through Vivado. Finally, the design is deployed onto the ZU4EV development board for board validation

The entire flow focuses on three core questions

- Whether the decoding result is correct
- Whether the hardware resources are acceptable
- Whether the timing and board-level interface of the design can run through on the target platform

Functional verification has already passed in a closed loop. All 6 unit tests of the Python model passed, namely

- K=3 no-noise smoke test, where K represents the constraint length of the convolutional code, that is, how many input bits the encoder refers to when generating output. K=3 means the encoder refers to the current bit and the previous 2 historical bits, so the trellis has only $2^{K-1} = 4$ states, and each state is a Viterbi Decoder guess about a certain history. This test first checks the basic encoding and decoding flow with a smaller 4-state trellis, making it convenient to confirm that the state transition and path selection logic have no problems before entering the formal parameters
- K=3 soft3 no-noise smoke test, adding soft3 input under the small-scale K=3 configuration. soft3 refers to a 3-bit soft-decision symbol, that is, using a value from 0 to 7 to indicate whether the received symbol is closer to 0 or closer to 1. This test is used to confirm that the calculation method of the soft-decision branch metric is correct, and that the original input can be recovered when there is no noise
- K=7 formal no-noise test, where K=7 is the constraint length formally used in this project, meaning the encoder refers to the current bit and the previous 6 historical bits, so the trellis has $2^{K-1} = 64$ states. This

test verifies, under the formal 64-state configuration and without adding channel noise, whether the basic decoding result of the Viterbi Decoder is completely consistent with the golden output

- K=7 tail termination back to zero state test, where tail termination means appending several 0s at the end of the payload so that the encoder internal shift register returns to the all-0 state. This allows the decoder to know the ending state of the trellis and reduces uncertainty in the ending path. This test checks whether both the encoder and decoder can correctly handle the zero state termination condition after zero padding at the end of the frame
- K=7 single encoded bit error correction test, where under the formal K=7 configuration, one bit after encoding is artificially flipped to simulate a single error occurring during transmission. The goal of the test is to confirm whether Viterbi Decoder can use the redundancy information of convolutional codes to still recover the correct original payload when one encoded bit error exists
- K=7 soft3 finite bit width plus subtract-min normalization test, where this test uses the formal K=7 configuration, 3-bit soft-decision input, finite path metric bit width, and subtract-min normalization. Finite bit width means the path metric in hardware cannot grow infinitely and can only be stored using a fixed number of bits; subtract-min normalization subtracts the current minimum value from all path metrics simultaneously at each step, preserving relative magnitude relationships while avoiding numerical overflow. This test is used to confirm that after adding these hardware implementation constraints, the decoding result is still correct

The limitations also need to be clearly stated. During the Vivado synthesis and implementation stage, the soft-decision hero design performed placement and routing with a target frequency of 100 MHz, but timing closure was not completed. The post-route WNS was -20.433 ns. WNS is Worst Negative Slack, indicating how much time the worst timing path still lacks to satisfy the target clock; a negative value indicates that there is a critical path that cannot complete within the target clock cycle. 100 MHz corresponds to a 10 ns clock period, so the current design cannot yet be called a 100 MHz timing-clean design.

During the board-level verification stage, after the board shell clock was reduced to 25 MHz, the design could pass functional verification on the real ZU4EV development board. This indicates that the decoding logic, PS/PL data transfer, and UART verification flow of the current architecture are correct, but more accurately, it is currently a functionally correct board-level verification version, not a high-frequency implementation version that satisfies the 100 MHz timing target. If 100 MHz is to be achieved later, the critical path needs further optimization. Pipeline can be added in the ACS computation path, splitting the addition, comparison, and selection originally completed within one clock cycle into multiple clock cycles; retiming can also be used to adjust register positions so that combinational logic is distributed more evenly; in addition, the structure of the ACS array can be reorganized to reduce the long combinational path pressure produced when 64 states are updated simultaneously

1. Glossary

The table below explains the core English terms that first appear in this report, for reference and lookup

Table 1：Core glossary.

Term	Explanation
Resource-Aware	Resource-aware, meaning that LUT, FF, BRAM, DSP, power, and timing are considered together during design, rather than focusing only on functional correctness.
Soft-Decision	Soft-decision, where the receiver does not only give 0/1, but also gives confidence. This project uses a 3-bit soft symbol to indicate whether the received value is closer to 0 or 1.
Hard-Decision	Hard-decision, where the receiver only gives 0 or 1, and the decoder cannot use confidence information.
VLSI / Very Large Scale Integration	Very large scale integration, emphasizing mapping algorithms into hardware implementations constrained by area, power, frequency, and timing.
LUT / Look-Up Table	Look-up table, the basic resource for implementing combinational logic in FPGA.
FF / Flip-Flop	Flip-flop, the basic register resource used in FPGA to store state on clock edges.
Convolutional Code	Convolutional code, an error-correcting coding method with memory, where the current output is determined by both the current input and historical inputs.
Trellis	Trellis, representing all possible state transition paths of a convolutional code by unfolding them over time.
Branch Metric	Branch metric, representing the gap between the expected output of a certain state transition branch and the actual received symbol.
Path Metric	Path metric, representing the total accumulated cost of a candidate path from the beginning to the current moment.
ACS / Add-Compare-Select	Add-Compare-Select unit, which first adds the branch metric to candidate paths, then compares the path metrics, and finally selects the path with smaller cost.
Traceback	Traceback, recovering the most likely input bit sequence backward from the endpoint according to survivor information.
Subtract-Min Normalization	Subtract-min normalization, subtracting the current minimum value from all path metrics simultaneously to control the numerical range and prevent path metrics from growing without bound.

Term	Explanation
BMU / Branch Metric Unit	Branch metric unit, responsible for calculating the distance between received symbols and the expected output of each trellis branch.
Survivor RAM / Survivor Random Access Memory	Survivor path memory, used to record which predecessor path each state selected at each time step.
ARM / Advanced RISC Machine	Embedded processor core, used in Zynq devices to run control programs and board-level test programs.
PS / Processing System	Processing system, the ARM processor and its peripheral part in Zynq.
PL / Programmable Logic	Programmable logic, the FPGA region in Zynq used to implement custom hardware circuits.
AXI DMA / Advanced eXtensible Interface Direct Memory Access	AXI direct memory access module, used to move data between PS memory and PL data stream interfaces.
UART / Universal Asynchronous Receiver/Transmitter	Serial communication interface, used to print board-level test logs to the computer.
Bitstream	FPGA configuration file, used to define the final hardware circuit built in the PL region.
ELF / Executable and Linkable Format	Executable program file, referring in this project to the bare-metal test program running on the ARM processor.
JTAG / Joint Test Action Group	Debugging and download interface, used to download bitstream and ELF to the development board and perform hardware debugging.
OCM / On-Chip Memory	On-chip memory, a shared storage region inside Zynq with small capacity but fast access speed.
WNS / Worst Negative Slack	Worst negative slack, indicating how much time the worst path still lacks to satisfy the target clock; a negative value indicates timing failure.
TNS / Total Negative Slack	Total negative slack, indicating the sum of negative slack across all timing-failing paths.
BRAM / Block RAM	Block RAM, a larger-granularity on-chip RAM resource inside FPGA.
DSP / Digital Signal Processing Slice	Digital signal processing hard block, a hardware resource inside FPGA specialized for multiplication, addition, and multiply-accumulate operations.
Power	Power, indicating the power budget consumed when the design runs. The Vivado power in this report is a tool estimate.

Term	Explanation
BER / Bit Error Rate	Bit error rate, indicating the ratio of erroneous bits to total bits. This report only reports observed mismatch rate based on limited test vectors, and does not claim to obtain a complete statistical BER curve.
AWGN / Additive White Gaussian Noise	Additive white Gaussian noise, a random noise model commonly used in communication systems to generate noisy test inputs.
Case	Test case, a verification sample consisting of a set of input, expected output, and metadata.
Mismatch	Mismatched bit, indicating a bit where the decoding output and golden output are inconsistent.
Run ID	Run identifier, used to identify a specific experiment or board-level test record.
Hero Design	Main design version, referring in this project to the final 3-bit soft-decision Viterbi decoder focused on optimization and board validation.
Board Shell	Board-level shell, the PS, DMA, registers, clock, and logging system wrapped outside the decoding core.
DDR / Double Data Rate Memory	External dynamic memory, with larger capacity but a more complex access path than OCM.
Pipeline / Retiming	Pipeline / retiming, shortening combinational logic paths by inserting registers or adjusting register positions, thereby improving timing.
SNR / Signal-to-Noise Ratio	Signal-to-noise ratio, representing the ratio between signal strength and noise strength.
AI tools	AI tools, used to assist planning, script generation, debugging organization, and report drafting, while final verification and conclusions are the responsibility of the author.

2. Technical Background

Communication and storage systems encounter noise in real operation. Noise causes the data seen by the receiver to be not completely consistent with the data originally sent by the transmitter, for example some bits are flipped, or the received signal becomes insufficiently certain. The most direct solution is retransmission, but retransmission is not always feasible. For example

- Deep-space communication has very long distances and long round-trip waiting times. For example, satellites, probes, and ground stations may be extremely far apart. If every error relies on retransmission,

signal round trips may take minutes or even longer, so error correction must be performed directly at the receiver as much as possible

- Low-power wireless nodes have limited energy and cannot retransmit frequently. For example, sensor nodes and IoT devices are usually battery-powered, and every wireless transmission consumes energy. Frequent retransmission significantly shortens device lifetime, so error-correcting decoding is needed to reduce the number of retransmissions
- Real-time video links are sensitive to latency and cannot always wait for erroneous data to be resent. For example, in live streaming, drone video transmission, or video conferences, data must arrive continuously and with low latency. If retransmission is repeatedly requested, the image will freeze or latency will increase, so it is more suitable to restore erroneous data as quickly as possible through forward error correction
- High-speed on-board data channels may also be unable to repeatedly transmit the same segment of data due to bandwidth and timing limitations. In FPGA or SoC internal high-speed data streams, data is usually transmitted continuously at a fixed rhythm. If frequent rollback and retransmission occur, extra bandwidth is occupied and pipeline timing is disrupted, so the decoder needs to complete error correction directly in the data stream

  Therefore, the value of error-correcting codes lies in this: the transmitter adds a certain amount of redundancy in advance, allowing the receiver to have a chance to recover the original data without retransmission.

This project chooses Viterbi Decoder because it can reflect both the basic idea of error-correcting algorithms and the resource and timing pressure in hardware implementation [1], [2]. Its core problem can be summarized in one sentence: after the receiver obtains a sequence of encoded bits that may have been polluted by noise, how to recover the most likely original input bit sequence

A convolutional code cannot be directly reverse-looked-up bit by bit like ordinary coding. The reason is that each group of encoded bits is not determined only by the current input bit, but is also related to the historical bits stored inside the encoder. Taking K=3 as an example, K=3 means that every time the encoder outputs, it refers to the current input bit and the previous 2 historical bits. Therefore, the encoder internally needs to remember the previous 2 historical bits, and the combination of these two historical bits is called the state. For example, state 00 means that the two historical bits currently remembered by the encoder are both 0; state 10 means that the most recent historical bit combination is 1 and 0

The following small example explains why direct reverse lookup is impossible. Suppose according to the convolutional encoding rule,

```
output[0] = current input bit XOR previous 1 historical bit XOR previous 2 historical bits
output[1] = current input bit XOR previous 2 historical bits
```

Assume the initial state is `00` , and the original input is `100` , then this input makes the state change in the following order

At the first clock, input 1, current state is 00

```
output[0] = 1 XOR 0 XOR 0 = 1
output[1] = 1 XOR 0 = 1
```

So the first group of encoded bits is 11. Input 1 is pushed into the shift register, and the original historical bits move backward, so the next state becomes 10

At the second clock, input 0, current state is 10, indicating that the previous two historical bits are 1 and 0：

```
output[0] = 0 XOR 1 XOR 0 = 1
output[1] = 0 XOR 0 = 0
```

So the second group of encoded bits is 10. After input 0 is pushed into the shift register, the next state becomes 01.

At the third clock, input 0, current state is 01, indicating that the previous two historical bits are 0 and 1：

```
output[0] = 0 XOR 0 XOR 1 = 1
output[1] = 0 XOR 1 = 1
```

So the third group of encoded bits is 11. After input 0 is pushed into the shift register, the next state becomes 00.

Therefore, the state path corresponding to original input 100 is

```
00 -> 10 -> 01 -> 00
```

The encoded bits corresponding to this state path are：

```
11 10 11
```

If the channel has no noise, the receiver also receives `11 10 11`, and decoding is relatively simple. But if the second group of encoded bits becomes erroneous during transmission, what the receiver actually receives is

```
11 00 11
```

At this point it is impossible to simply reverse-look-up each group of received bits group by group into original bits. Because the middle `00` may really be the result that some path should output, or it may be the result of the original `10` being corrupted by noise. The receiver does not know which bit was affected by noise, so it needs to judge which kind of original input is most reasonable from the perspective of the entire path.

The method of Viterbi Decoder is to expand all possible state paths into a trellis. Each branch in the trellis represents one possible input choice, that is, input `0` or input `1`. As long as the current state and input bit are determined, the next state and the encoded bits that should be output at this clock are also determined.

Therefore, the decoder can perform a trial encoding for each candidate path, and then compare the encoded bits produced by the trial with the received bits actually received.

In the example above, the candidate path `00 -> 10 -> 01 -> 00` corresponds to the expected output：

```
11 10 11
```

Due to noise, the actual received sequence is

```
11 00 11
```

After group-by-group comparison, only the second group differs by 1 bit, so the accumulated cost of this path is：

```
0 + 1 + 0 = 1
```

If the expected output of another candidate path is：

```
11 01 01
```

Its distance from the actual received sequence `11 00 11` is：

```
0 + 1 + 1 = 2
```

Therefore, the accumulated cost of the first path is smaller, indicating that it is more likely to be the real transmitted path. Viterbi does not require the path to be exactly the same as the received result, but finds the “most similar” path. Because the received result may be polluted by noise, directly reverse-looking-up according to the received data may instead be wrong. Viterbi assumes that there may be a small number of errors in the channel, so it compares all legal paths, selects the path with the minimum distance to the received sequence, that is, the path with the minimum path metric, and reads the corresponding input bit from each branch on the path to recover the decoded bits

```
100
```

If channel noise is small, the correct path is usually closer to the received sequence than the wrong path, so the original data can be recovered; if noise is too large, a wrong path may also be closer to the received sequence than the correct path, in which case the decoder may still make an error. Therefore, Viterbi Decoder does not guarantee error correction under all conditions, but under the redundancy constraints provided by convolutional codes, selects the input path that is most likely in the maximum-likelihood sense

From a hardware perspective, the challenge of Viterbi Decoder is that this process needs to be repeated continuously. BMU（Branch Metric Unit）is responsible for calculating the gap between each branch and the

received data; ACS（Add-Compare-Select）is responsible for adding the new branch metric to the existing path metric, and selecting the smaller-cost path among multiple candidate paths; survivor recording is responsible for saving which predecessor path each state selected at the current moment; traceback is responsible for tracing backward according to these survivor records to recover the final input bit sequence.

Therefore, the focus of this project is not only to implement a Viterbi Decoder that can decode, but also to study how it can be implemented more reasonably on FPGA. The same Viterbi algorithm can choose hard-decision or soft-decision input, and can choose different traceback depth, path metric width, and normalization methods. These choices directly affect decoding correctness, LUT/FF resource usage, timing closure, and power consumption. In other words, the algorithmic structure of Viterbi Decoder is very clear, but when it is truly implemented in hardware, tradeoffs need to be made among correctness, resources, frequency, and board-level verification.

3. Design Specification

The problem this project aims to solve can be summarized as: implement a Viterbi Decoder capable of processing convolutional codes on the ZU4EV FPGA platform, and verify whether it can maintain consistent decoding results across the software model, RTL simulation, Vivado implementation, and real development board operation. The design parameters are not manually scattered throughout the report, but uniformly come from `spec/viterbi_spec.json` . This ensures that the Python model, test vector generation, RTL package, and later verification flow use the same set of configuration, avoiding parameter inconsistency across different stages.

The formal convolutional code configuration adopted by this project is rate-1/2, K=7, generator polynomials `[171,133]` [3]. rate-1/2 means that for every 1 original input bit, the encoder outputs 2 encoded bits, so the transmitter adds one-times redundancy information. K=7 means the constraint length is 7, that is, every time the encoder outputs, it refers to the current input bit and the previous 6 historical bits. Since the state only needs to store historical bits and does not include the current input bit, the number of states is $2^{K-1} = 2^6 = 64$:

$$2^{K-1} = 2^6 = 64$$

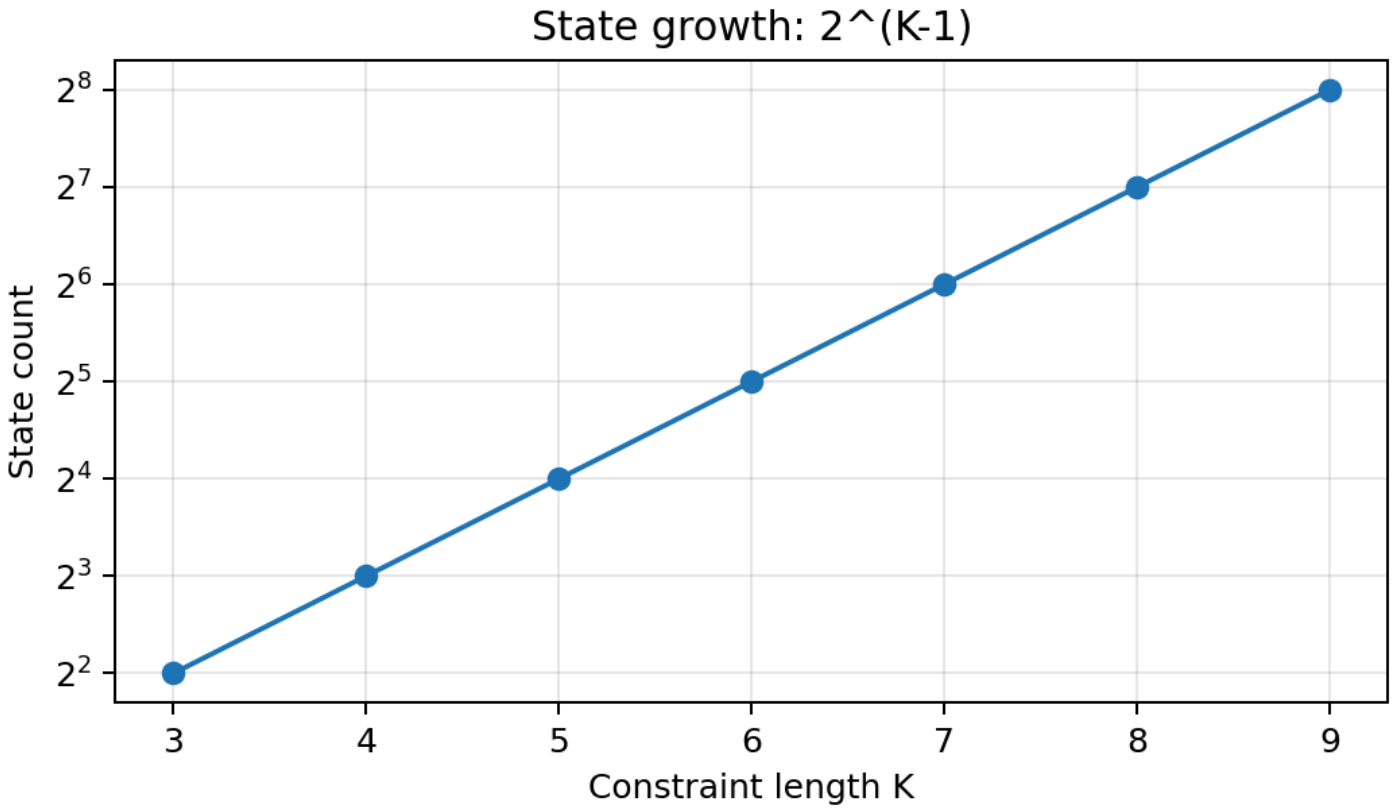

Figure 1：The number of states grows exponentially with K. This figure shows that when constraint length increases, the number of trellis states grows exponentially. K=7 corresponds to 64 states, which means that the subsequent BMU, ACS, survivor memory, and traceback all need to be organized around a 64-state trellis. Hardware scale, resource usage, and timing pressure are all significantly higher than in the 4-state small example

`[171,133]` are the two generator polynomials of the convolutional encoder, represented in octal. Since this project uses K=7, every time the encoder outputs, it refers to the current input bit and the previous 6 historical bits, for a total of 7 bits. Octal 171 converted to binary is 1111001, meaning the first output selects register bits at positions corresponding to 1 for XOR; octal 133 converted to binary is 1011011, meaning the second output selects another group of register bits for XOR. Therefore, for every 1 original input bit, the encoder generates two encoded bits separately according to these two XOR tap rules, which is also the source of rate-1/2

This project keeps both a hard-decision baseline and a soft-decision hero design. The baseline is the hard-decision version, where the input only contains 0 or 1, mainly used to verify whether the trellis construction, state numbering, and basic ACS flow are correct. The hero design is the final 3-bit soft-decision version focused on verification, where the input uses 0 to 7 to indicate the confidence of the received symbol, and can preserve more channel information than hard-decision.

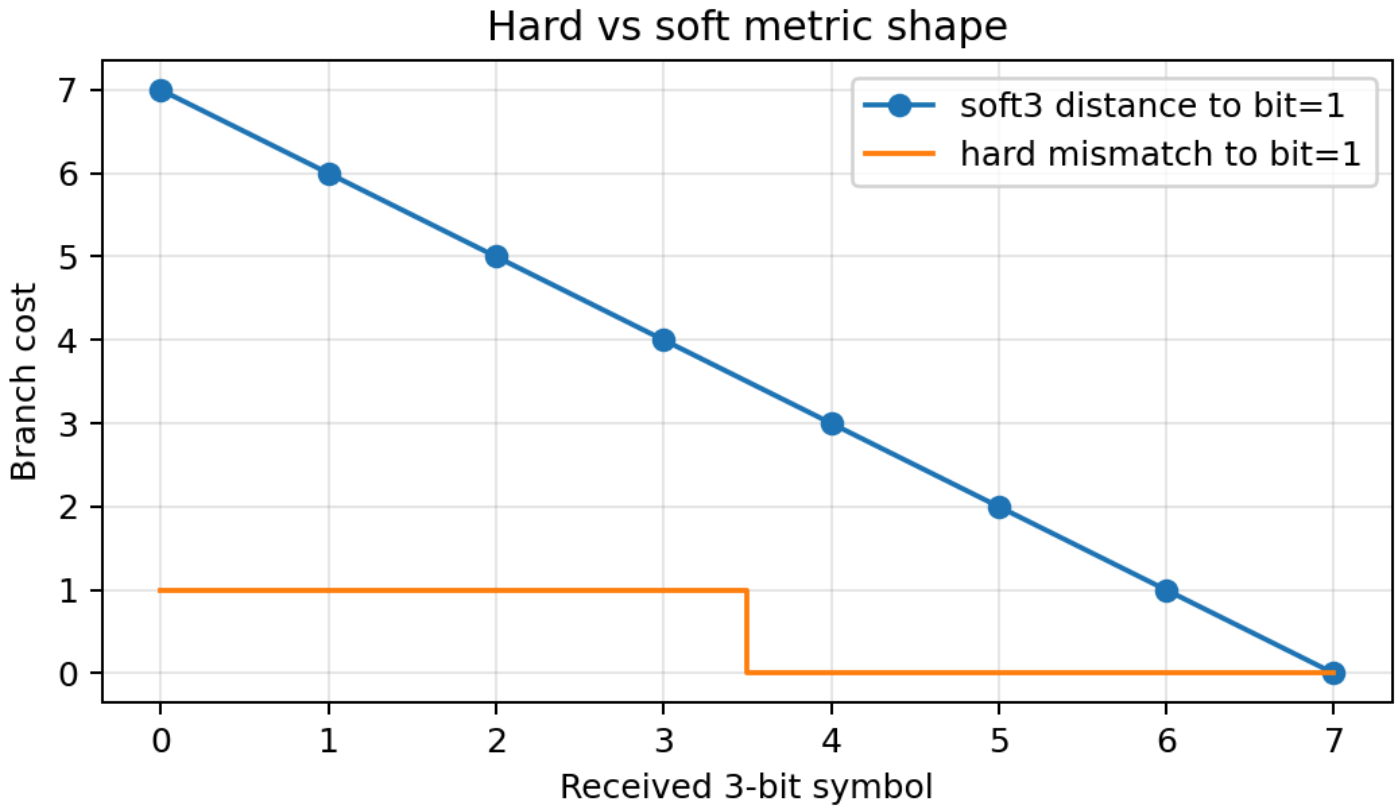

Figure 2：Shapes of hard-decision and soft-decision branch costs. Hard-decision first directly decides the received signal into 0 or 1, so the branch cost only reflects a coarse judgment of “whether they are the same”; for example, when receiving 1, it only knows that it was decided as 1, but does not know whether this 1 is very reliable or close to the decision boundary. Soft-decision preserves the distance between the received value and ideal 0 or ideal 1. This project uses a 3-bit soft symbol to represent a confidence scale from 0 to 7, so the decoder can use finer reliability information when comparing candidate paths. In this way, even if two candidate paths look similar under hard-decision, soft-decision may select a more reasonable path based on the distance differences of the received values.

For the hero design, the core parameters are: traceback depth set to 40, path metric width set to 12 bit, and subtract-min normalization used to control the numerical range of the path metric. These parameters are determined through parameter sweep. By separately changing key design parameters and observing the mismatch count and hardware cost under different configurations, a configuration more balanced among correctness, resources, and latency is selected.

- traceback depth：Traceback depth, that is, how many steps to trace backward before determining the output bit during traceback. If the depth is too small, the path may not have fully converged and can easily be selected incorrectly; if the depth is too large, decoding is usually more stable, but it increases survivor memory requirements and waiting time.
- path metric width：Path metric bit width, that is, how many bits are used in hardware to store accumulated path cost. If the width is too small, the path metric may overflow or truncate, causing incorrect path comparison; if the width is too large, it increases the register resources and timing pressure corresponding to 64 states.

- normalization：Normalization method, used to control the problem that path metrics grow larger and larger due to continuous accumulation. This project uses subtract-min normalization, that is, at each step, all path metrics simultaneously subtract the current minimum value. This does not change the relative magnitude between paths, but can reduce the numerical range and decrease overflow risk.

The project tried different traceback depths, different path metric widths, and different normalization schemes, and used the same batch of test vectors to count mismatch numbers. The results show that under the current test vectors, when traceback depth is 40, path metric width is 12 bit, and subtract-min normalization is used, 0 mismatch can be achieved, while unnecessary storage, latency, and resource pressure are not further increased as they would be with larger depth or larger bit width. Therefore, this set of configuration is selected as the final hero design.

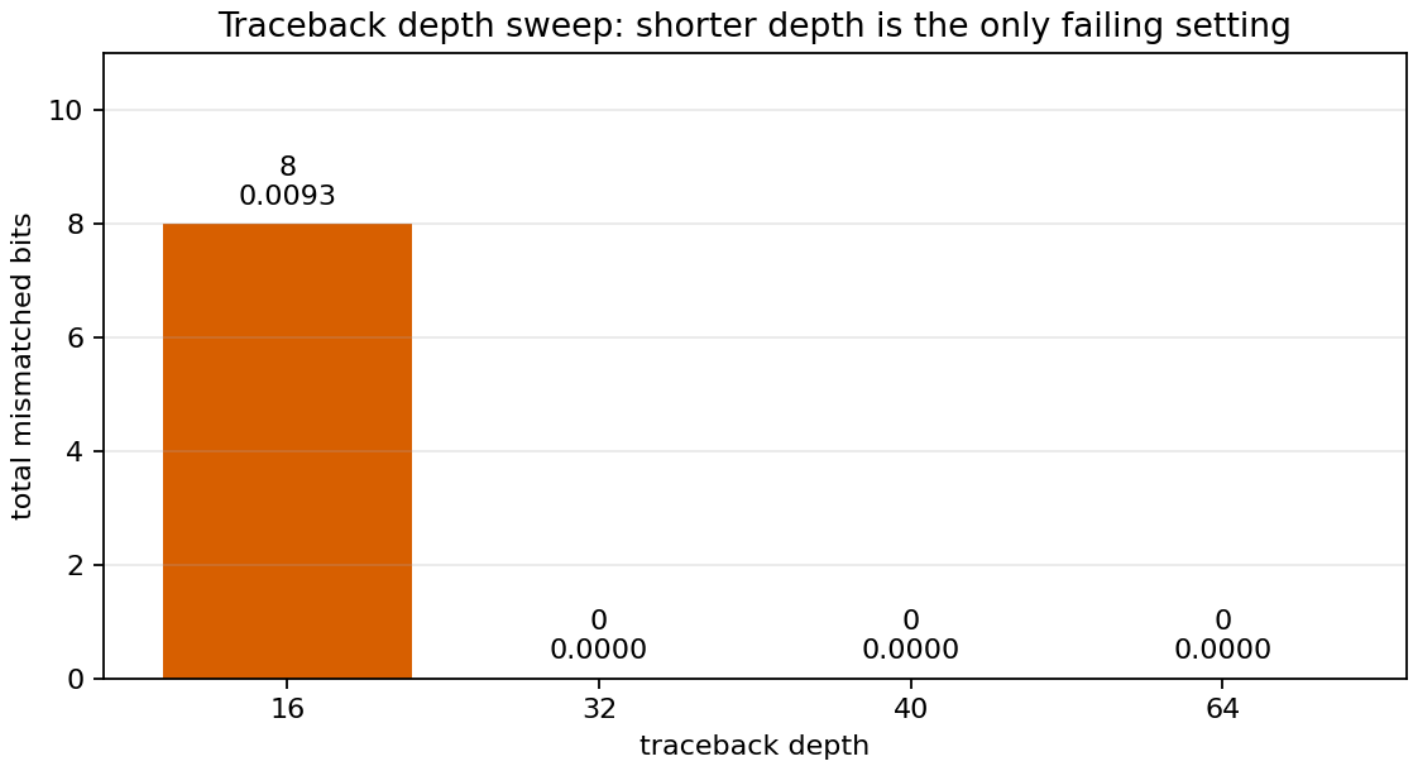

Figure 3：Traceback depth sweep. This figure shows the influence of traceback depth on mismatch count. When depth=16, 8 mismatches appear in the current vector set, indicating that when traceback depth is too short, the path has not fully converged, and the decoder may make judgments too early. When depth=32, 40, 64, the mismatch is 0 under the current test vectors. Therefore, the meaning of depth=40 is: it has already reached 0 mismatch under the current test conditions, while reducing part of survivor memory requirements and traceback waiting time compared with depth=64.

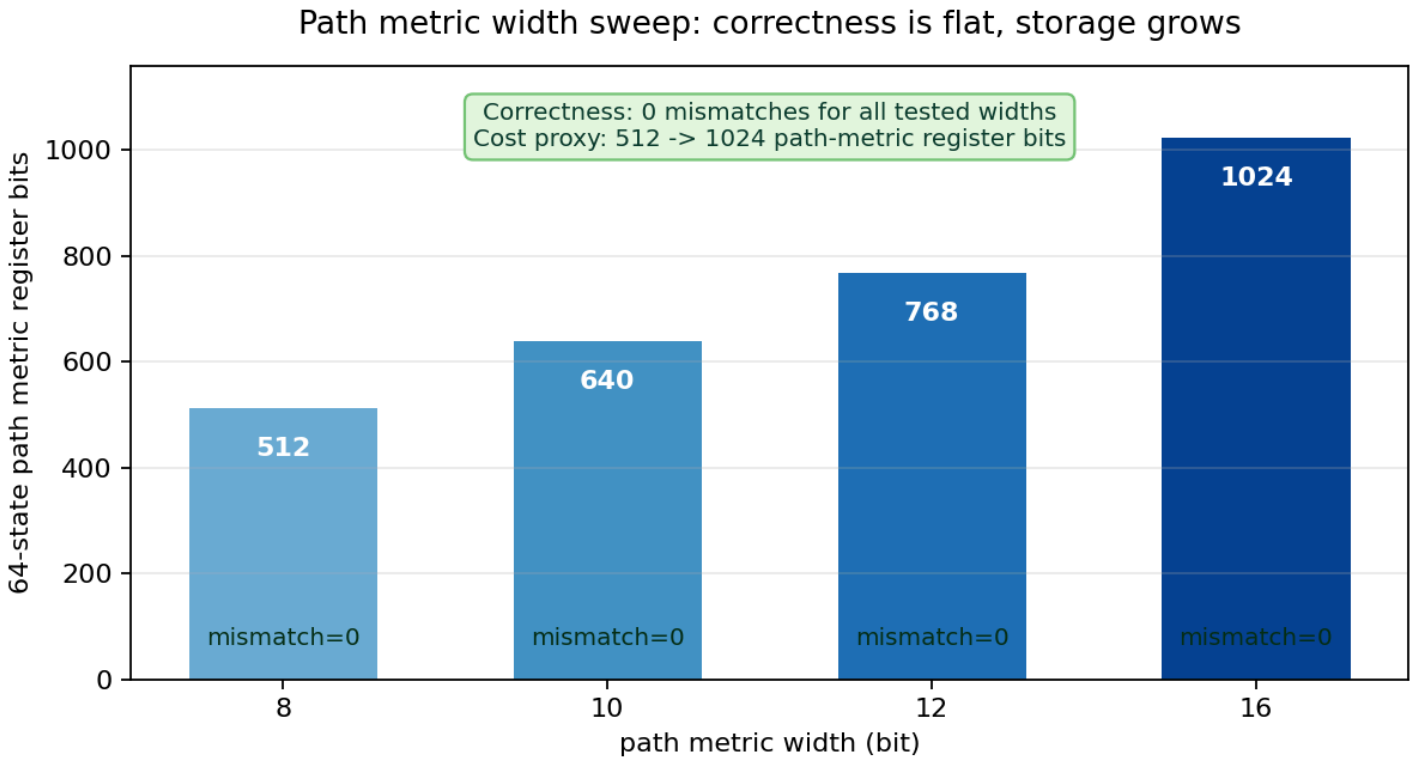

Figure 4：Path metric width sweep. This figure shows the influence of path metric width on hardware cost. Under the conditions of depth=40 and subtract-min normalization, no mismatch is observed in the current test set for 8/10/12/16 bit, so the figure directly marks mismatch=0 at the bottom of each bar; the bar height represents the total number of path metric register bits needed by 64 states. The larger the width, the more the register bits increase from 512 to 1024, and adders and comparators also become wider, increasing both resource usage and timing pressure. Therefore, choosing 12 bit is not because 16 bit is incorrect, but because 12 bit is already sufficient to pass the current functional closed loop while being more restrained than 16 bit.

Table 2：Design specification and parameter selection.

Project item	Value	Data source	Why this is selected
Coding configuration	Convolutional Code, rate-1/2, K=7, [171,133]	`spec/viterbi_spec.json`	rate-1/2 provides redundancy for error correction; K=7 produces 64 states, which can reflect Viterbi hardware structure pressure; [171,133] is a classical generator polynomial configuration.

Project item	Value	Data source	Why this is selected
baseline design	Hard-Decision Viterbi Decoder	`spec/viterbi_spec.json`	The input only has 0/1, and the structure is simpler, making it suitable as a base version to check whether trellis, state numbering, BMU, and ACS are correct.
hero design	3-bit Soft-Decision Viterbi Decoder	`spec/viterbi_spec.json`	soft-decision preserves received confidence, is more suitable than hard-decision for noisy input, and is the final version focused on optimization and board validation in this project.
soft symbol bit width	3 bit, value range 0 to 7	`spec/viterbi_spec.json`	Uses finite bit width to express whether the received symbol is closer to 0 or 1, balancing information amount and hardware resources.
traceback depth	40	`spec/viterbi_spec.json` ; `data/analysis/traceback_depth_sweep.csv`	Too short a depth can easily cause the path to not yet have converged; depth 40 reaches 0 mismatch in the current sweep vectors, while having lower latency than depth 64.
path metric width	12 bit	`spec/viterbi_spec.json` ; `data/analysis/path_metric_width_sweep.csv`	path metric needs to be wide enough to avoid accumulated cost overflow; 12 bit

Project item	Value	Data source	Why this is selected
			maintains 0 mismatch in the current test and avoids excessive resource increase.
normalization	Subtract-Min Normalization	`spec/viterbi_spec.json`	At each step, all path metrics subtract the current minimum value, preserving relative path magnitudes while controlling numerical growth.
payload length	96 bit	`vectors/*/metadata.json`	Covers a complete frame test while keeping RTL simulation and board-level regression runtime short, facilitating rapid iteration.
target development board	MZU04A-4EV / XCZU4EV	`spec/viterbi_spec.json`； `config/local.env`	Consistent with the actually available ZU4EV development board and Xilinx 2024.1 toolchain, ensuring final real board-level verification can be performed.

4. Repository Structure and Source of Truth

The organizing principle of this repository is that code, data, and report should not be mixed together. Algorithm models are placed in the model directory, hardware implementation is placed in the RTL directory, real experiment results are placed in the data directory, and the report only references these results that have already been written to disk. The benefit of doing this is that every number in the report can be traced back to the corresponding file for checking, rather than relying only on textual description.

The top-level repository structure is as follows：

```
src/
|-- README.md                 Project entry description, used to quickly understand how to run and verif
|-- Report.md                 Markdown source file of this report.
|-- Report.pdf                Submission-version PDF rendered from Report.md.
|-- spec/                     Source of truth for design parameters, where K, code rate, polynomials, an
|-- config/                   Local toolchain, serial port, board, and GitHub address configuration.
|-- model/                    Python encoder, channel model, and golden decoder.
|-- vectors/                  Unified test vectors, each case has input, output, and metadata.
|-- rtl/                      RTL implementation of the Viterbi decoder core and board-level wrapper.
|-- tb/                       xsim testbench, used to align and compare RTL output with golden output.
|-- vivado/                   Vivado scripts for synthesis, implementation, and ZU4EV block design const
|-- vitis/                    ZU4EV bare-metal app, used for DMA transfer and UART printing.
|-- scripts/                  Helper scripts for automatic generation, simulation, synthesis, board run,
|-- data/                     All model, simulation, parameter sweep, Vivado, and board-level run result
|-- docs/                     Figures, flowcharts, and board images used in the report.
|-- reports/                  Final checks, PDF preview, and staged review files.
|-- WORKLOG.md                What commands were actually executed at each stage, what problems occurred
|-- DECISIONS.md              Records of key engineering tradeoffs, such as why OCM buffer was used.
`-- EXPERIMENTS.yaml          Experiment index, associating commands, results, and data files using run_
```

The most important among these are `spec/` and `data/`

- `spec/viterbi_spec.json` is the source of truth for design parameters
- `data/model/` records Python tests
- `data/regression/` records RTL simulation；
- `data/analysis/` records parameter sweeps and approximate BER analysis；
- `data/impl/` records Vivado resources, timing, and power estimates；
- `data/board_runs/` records real UART logs

5. End-to-End Engineering Flow

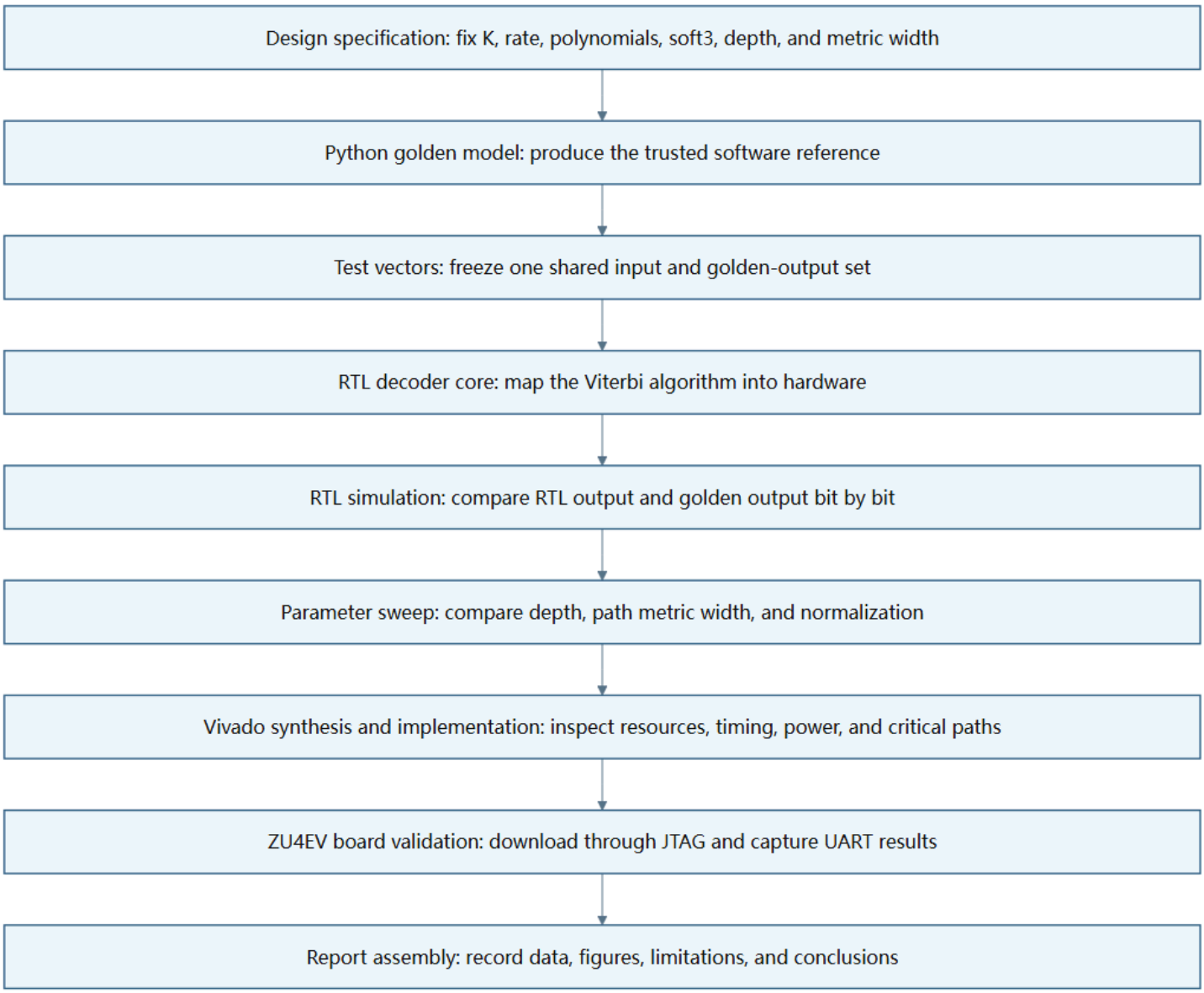

Figure 5：End-to-end engineering closed loop. This figure emphasizes what each step produces and why the next step needs it. The design specification first fixes the parameters, the Python model first establishes the standard answer, the test vectors fix the inputs and outputs, RTL simulation proves that the hardware logic is consistent with the software answer, parameter sweep explains why the current hero parameters are selected, Vivado tells us the resource and timing pressure, and board-level verification finally proves that data can really run through PS, DMA, PL, and UART.

The role of each step is as follows

- Design specification：Answers “what kind of Viterbi Decoder exactly are we going to make”. Without a unified specification, Python, RTL, and board app can easily each use a different set of K, tail termination, or soft symbol format, and in the end, even if the results are inconsistent, it will be difficult to locate the problem.

- Python golden model：Its role is not to pursue speed, but to pursue clarity and credibility. All subsequent RTL output and board output must align and compare with it.
- Test vectors：Like a unified exam paper, the same problem is given to Python, RTL simulation, and the real development board. In this way, during debugging, there is no need to guess whether the input is the same.
- RTL core：Implements BMU, ACS, path metric, survivor memory, and traceback all as hardware structures.
- Simulation：Not only looking at waveform, but comparing decoded bits with golden output bit by bit. Only after bit-true comparison passes can it be said that the function is really aligned.
- Parameter sweep：Compares the influence of traceback depth, path metric width, and normalization, and uses mismatch count to confirm hero selection rather than selecting parameters by intuition.
- Vivado synthesis and implementation：Answers “can a functionally correct design fit into FPGA, and can it run at the target clock”. This project found here that the soft-decision version did not reach 100 MHz timing closure.
- Board-level verification：Puts the design onto the real ZU4EV board, downloads bitstream and ELF through JTAG, and captures run logs through UART. It verifies the entire PS/PL/DMA data path, not just the decoder core.

6. Branch Metric, Soft-Decision, and Fixed-Point Path Metric

Earlier it was said that Viterbi finds the most similar path, so the branch metric here is the scoring method for similarity.

First look at hard-decision. Because this project is rate-1/2, for every 1 payload bit input to the encoder, it outputs 2 encoded bits. That is, every step on the trellis, the receiver obtains a group of 2-bit received bits. In hard-decision, each received bit is only 0 or 1, so the distance of a branch can only be three values: 0, 1, 2, namely $d_{\text{hard}} \in 0, 1, 2$.

For example, a certain candidate branch should theoretically output `10`. If the receiver also receives `10`, both bits are the same, and the branch metric is 0. This means this step matches completely.

```
expected = 10
received = 10
distance = 0
```

If the receiver receives `00`, the first bit is different and the second bit is the same, so the branch metric is 1. This means this step has 1 bit that does not match, but is not completely dissimilar yet.

```
expected = 10
received = 00
distance = 1
```

If the receiver receives `01`, both bits are different, so the branch metric is 2. This means this step is the least similar to this candidate branch.

```
expected = 10
received = 01
distance = 2
```

Therefore, hard-decision has exactly two encoded bits at each step. 0 means both are correct, 1 means one is wrong, and 2 means both are wrong. This is also why hard-decision information is relatively coarse: it only knows how many bits are wrong, but does not know how suspicious each bit error is.

The difference with soft-decision is that the receiver does not only give 0 or 1, but gives a 3-bit value, namely 0 to 7. Here, 0 can be understood as very similar to 0, 7 as very similar to 1, and middle values 3 or 4 indicate “not very certain”. Therefore, soft-decision can express confidence.

The soft3 branch metric of this project is calculated as follows: if the candidate branch expects a certain encoded bit to be 0, the ideal value is treated as 0; if it expects 1, the ideal value is treated as 7. Then the absolute difference between the received soft symbol and the ideal value is calculated. Because rate-1/2 has two encoded bits at each step, the sum of the two differences is the cost of this branch, and the formula can be written as $BM = |r_0 - \hat{c}_0| + |r_1 - \hat{c}_1|$.

For example, a certain candidate branch expects output `10`, that is, the first bit should ideally be like 1, and the second bit should ideally be like 0. If the received soft3 value is `(6,2)`, then the cost is：

```
expected bits = 1 0
ideal soft    = 7 0
received      = 6 2
distance      = |6-7| + |2-0| = 1 + 2 = 3
```

That is, $BM = |6 - 7| + |2 - 0| = 3$. The meaning of this 3 is that the first bit is very similar to 1, only differing by 1; the second bit deviates somewhat from 0, differing by 2; together, the local cost of this branch is 3. If another candidate branch expects output `00`, the ideal value is `(0,0)`, and with the same received `(6,2)`, the cost becomes：

```
expected bits = 0 0
ideal soft    = 0 0
received      = 6 2
distance      = |6-0| + |2-0| = 6 + 2 = 8
```

That is, $BM = |6 - 0| + |2 - 0| = 8$. Therefore, when receiving `(6,2)`, branch `10` is more reasonable than branch `00`. The advantage of soft-decision is here: it does not only see that 6 will eventually be hard-decided as 1, but also knows that it is “very like 1”; it also does not only see that 2 will eventually be hard-decided as 0, but also knows that it is still some distance away from 0.

Path metric accumulates each step's branch metric along the same candidate path. Branch metric only describes the gap between a certain state transition and the current received symbol, while path metric describes the accumulated gap between the entire path and the received sequence from the starting point to the current state. Each time the Viterbi Decoder moves one step, it adds the new branch metric to the existing path metric, then compares the total cost of different candidate paths. The smaller the total cost, the more the path overall fits the received data, and the more likely it is the real transmitted path

However, in hardware implementation, path metric cannot grow infinitely. In software, larger integer types can be temporarily used to store accumulated values, while the register bit width in FPGA must be fixed in advance. For example, if path metric width is set to 12 bit, then it can only represent values within a finite range. The longer a frame is, the more branch metric accumulation times there are, and the more likely path metric is to become large. If no control is added, the numerical value may exceed the range representable by the fixed bit width, causing overflow or truncation. Once path metric overflows, the magnitude relationship between paths may be destroyed, and ACS may select the wrong survivor path.

To solve this problem, this project uses subtract-min normalization. Its idea is: what Viterbi Decoder truly cares about is not the absolute cost of each path, but which path is smaller than another path. In other words, path selection only depends on relative magnitude, not absolute numerical value. For example, the path metrics of three paths are：

```
Path A = 105
Path B = 112
Path C = 130
```

The optimal path is Path A. If all paths simultaneously subtract the current minimum value 105, we get：

```
Path A = 0
Path B = 7
Path C = 25
```

It can be seen that the values as a whole become smaller, but the relative order of the three paths does not change. Path A is still the smallest, Path B is still second, and Path C is still the largest. Therefore, subtract-min normalization does not change Viterbi's path selection result, but it can compress path metric back into a smaller range and reduce the risk of fixed-width register overflow.

In the specific hardware flow, subtract-min normalization usually occurs after ACS update at each trellis step. The decoder first calculates the new path metric for all states, then finds the minimum value among these path metrics, and then makes all states' path metrics simultaneously subtract this minimum value. The formula is $PM'_s = PM_s - \min_i(PM_i)$. The result of doing this is that after each step, at least one state's path metric is normalized to 0, and other states store the extra cost relative to this optimal state. Since all states subtract the same number, it does not affect subsequent ACS comparison of which is smaller, but only controls the numerical range to be more suitable for hardware implementation.

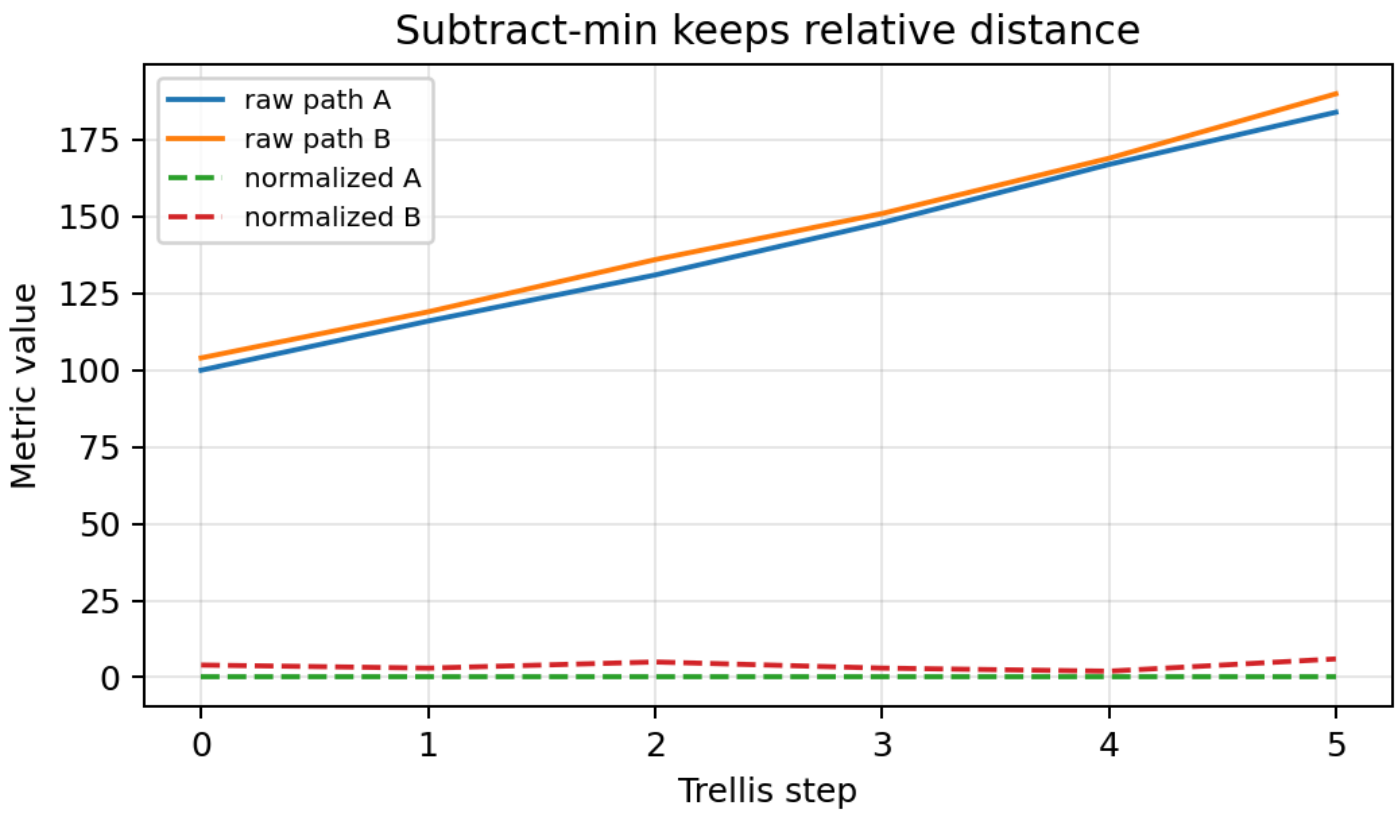

Figure 6：Subtract-min normalization. This figure shows the effect of subtract-min normalization. In the figure, the absolute path metric value of each path is overall reduced, but the relative magnitude between paths does not change. That is, whichever path is optimal before normalization is still optimal after normalization. It solves the numerical range and overflow risk problem in fixed-width hardware, rather than changing the path selection logic of Viterbi Decoder

7. RTL Architecture

The RTL architecture can be divided into two levels：

- decoder kernel：Viterbi decoding core, only responsible for the decoding algorithm itself, answering how the decoding result is calculated, focusing more on algorithm hardwareization, for example how branch metric is calculated, how 64 states are updated, how survivor decision is saved, and how traceback recovers bits
- board shell：The interface and control logic added outside the core for real board operation, including AXI Stream, AXI-Lite, DMA, PS/PL connection, and debug registers, answering how data is sent into the hardware and how results are taken back from the board, for example how the ARM-side program sends input data to PL, how DMA moves data, how control registers start the decoder, and how UART prints the final verification result

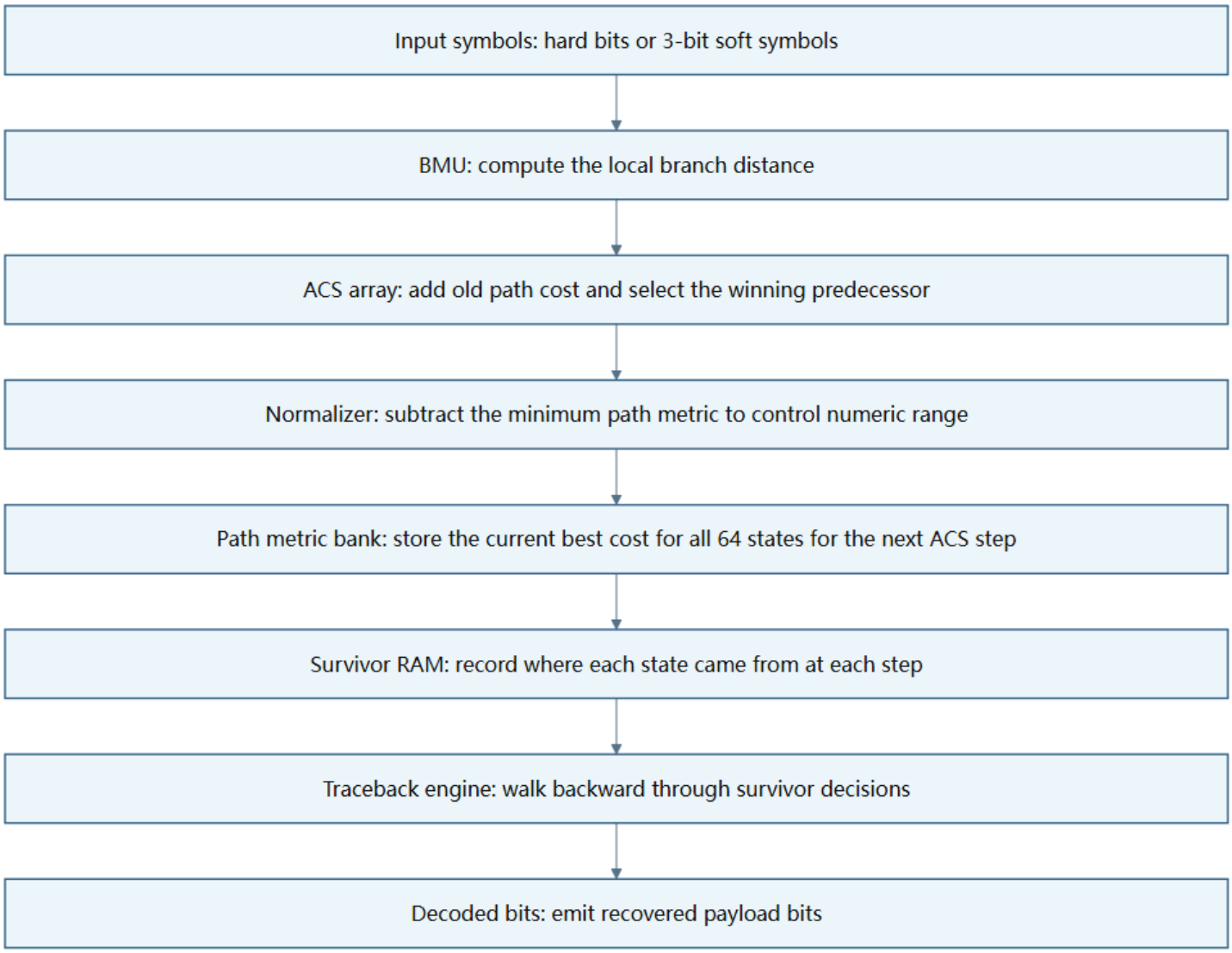

Figure 7：RTL decoding core structure. This figure shows the data flow inside the RTL decoding core. The received symbols first enter BMU, and BMU calculates the gap between each trellis branch and the received data; then the ACS array uses these branch metrics to update the optimal path metric of each state and select the most reasonable predecessor path for each state at the current moment; survivor RAM records these selection results; finally, the traceback engine traces backward according to the records saved in survivor RAM and outputs decoded bits. In other words, this figure corresponds to the hardware pipeline of Viterbi Decoder from “received symbols” to “recovered original bits”

The core modules are as follows

`bmu_hard.sv` is the hard-decision branch metric unit. It receives 2-bit hard received bits, compares them with the expected bits of each candidate branch, and outputs three distances: 0, 1, and 2.

`bmu_soft3.sv` is the soft-decision branch metric unit. It receives two 3-bit soft symbols, maps expected 0 to 0 and expected 1 to 7, and then calculates the sum of two absolute differences.

`acs_unit.sv` is a single Add-Compare-Select unit. It is responsible for selecting between two candidate paths: first adding the old path metric to the branch metric, then comparing which one is smaller, and finally outputting

the winner and survivor bit.

`acs_array.sv` is a 64-state ACS array. Because K=7 produces 64 states, the optimal path of 64 states needs to be updated at every trellis step.

`path_metric_bank.sv` stores the current path metric of all states. It can be understood as the “current optimal cost ledger” for each of the 64 states.

`path_metric_normalizer.sv` performs subtract-min normalization. It finds the minimum value among 64 path metrics and then subtracts this minimum value from all path metrics simultaneously, preventing fixed-width numerical overflow.

`survivor_ram.sv` stores the survivor decision of each state at each time step. It does not store the full path, but stores “which predecessor this state came from at that time”, and later traceback uses these records to find the path backward.

`traceback_engine.sv` is responsible for tracing paths backward from survivor RAM. Its output is not branch metric, but the final recovered decoded bits.

`viterbi_decoder_core_soft3.sv` is the top level of the soft-decision hero core. It connects BMU, ACS, normalization, survivor RAM, and traceback together to form a complete decoder.

Board-level related RTL also needs to be explained

`viterbi_axis_wrapper.sv` wraps the core into AXI Stream input/output form, allowing DMA to directly send in soft3 symbols and take back decoded bits

`viterbi_control_regs.v` provides AXI-Lite control registers, including input length, status, cycle count, seen/emitted counts, and other debug information

`viterbi_zu4ev_shell.v` connects the decoding core into the ZU4EV PS/PL system.

8. Verification Method

The verification principle is very simple: output bits must be aligned bit by bit with golden output. Waveform is only used to locate errors, such as checking whether TLAST, valid, ready, or survivor writes are abnormal; the final PASS/FAIL must come from bit-true comparison.

- Python model：First use the K=3 small-scale trellis for smoke test, because K=3 has only 4 states and errors are easier to locate; then switch to the formal K=7 configuration. There are 6 Python unit tests in total, covering no-noise, soft3 no-noise, tail termination, single-bit error correction, and finite bit width plus subtract-min normalization. The result is that all 6 passed.
- Test vectors：Fix no_noise, all_zero, impulse_one, single_bit_error, burst_error_short, random_hard, soft_awgn_0db, soft_awgn_1db, soft_awgn_2db into files. Each case has metadata, input, received

symbols, and golden output. In this way, RTL and board do not need to regenerate input randomly, avoiding “software tested one problem, hardware ran another problem”.

- hard-decision RTL：The hard baseline first runs no_noise, all_zero, impulse_one, then single_bit_error. This stage mainly confirms that trellis state numbering, branch metric, ACS, and traceback have no basic errors. The result file is `data/regression/rtl_regression_summary.csv` , and all 4 cases are 0 mismatch.
- soft-decision RTL：The soft3 hero continues to run soft_awgn_0db, soft_awgn_1db, soft_awgn_2db. This stage focuses on checking soft symbol packing, soft branch metric, finite path metric width, and subtract-min normalization. The result file is `data/regression/soft3_regression_summary.csv` , and all 3 cases are 0 mismatch.
- Parameter sweep：Does not change test vectors, only changes traceback depth, path metric width, and normalization. It answers “why the final parameters are selected this way”. The sweep results have already been shown in Figure 3 and Figure 4 in the design specification section.
- Vivado：No longer checks decoded bits, but checks whether RTL can be synthesized and routed, and what the resource, timing, and power estimates are. It exposes that the current soft-decision hero fails timing at 100 MHz.
- Real ZU4EV board：Checks not only the core, but also the PS, DMA, OCM buffer, AXI Stream packing, TLAST, cache flush/invalidate, and UART log. The latest passing UART run shows that all 3 cases PASS.

The verification results can be summarized as：

Table 3：Verification result overview.

Verification level	Main purpose	Result
Python model	Establish golden reference	All 6 unit tests passed
Test vector generation	Fix unified input and expected output	All 9 cases generated metadata and golden output
hard-decision RTL	Check basic trellis, BMU, ACS, traceback	All 4 cases 0 mismatch
soft-decision RTL	Check soft3, finite bit width, and normalization	All 3 AWGN cases 0 mismatch
Parameter sweep	Confirm hero parameter selection	depth=40, PM width=12, subtract-min passed
Vivado	Check resources, timing, and power estimates	Synthesis and implementation completed, but 100 MHz timing failed
ZU4EV board	Check real PS/PL/DMA/UART data path	All 3 cases 0 mismatch

Overall, the verification flow has already covered the main links from software reference model to real board-level operation, and can prove that the current design is functionally self-consistent: the same batch of input data, after passing through Python, RTL simulation, and the ZU4EV development board, can all align with

golden output. At the same time, the verification results also expose the boundary of the current design. The functional closed loop has already run through, but high-frequency timing is still the focus of future optimization. Therefore, the conclusion of this stage is that the current version has reached the functional goals of being verifiable, reproducible, and able to run on board, but critical paths still need further optimization before it can become a final implementation satisfying higher clock frequency requirements.

9. Simulation and BER Analysis

Here BER needs to be explained first. BER is Bit Error Rate, usually defined as the number of erroneous bits divided by the total number of bits. In communication systems, a strict BER curve generally requires a large number of random input bits, multiple SNR points, and sufficiently large statistical samples at each SNR point. The reason is that bit errors themselves are random. If the sample size is too small, having 0 mismatch in one test does not mean the true BER under that channel condition is 0; similarly, having a small number of mismatches in one test may also be affected by the specific random vector. Therefore, a complete BER curve is usually estimated through large-scale Monte Carlo simulation or long-frame testing.

The current project focuses on the end-to-end hardware closed loop, not large-sample BER modeling at the communication theory level. The payload of each case is 96 bit, and the sample size is small, mainly used to verify whether the Python model, RTL core, Vivado implementation, and board validation are consistent. Therefore, this report does not package these limited vector results as a complete communication BER curve. A more accurate term is observed mismatch rate, that is, the proportion of erroneous bits in the current limited test vectors. It reflects “how many bits are inconsistent between decoding output and golden output in this fixed batch of test samples”, rather than “the statistical BER of this decoder under all random channel conditions”.

The simulation results are viewed in three layers.

- no_noise：The meaning of the no-noise case is to check basic functionality. Under no-noise conditions, the encoded bits obtained by the receiver are exactly the same as the encoded bits generated by the transmitter, so the decoder should be able to stably recover the original payload. If no_noise fails, the problem is usually not channel noise, but basic logic such as trellis construction, state numbering, tail termination, branch metric, ACS, or traceback. In other words, the no-noise case is the lowest threshold of the entire verification flow. Only after it passes does it make sense to continue discussing soft-decision, AWGN, or mismatch rate.
- Artificial errors：Cases such as single_bit_error and burst_error_short are used to check whether the redundancy of convolutional codes is truly used by the Viterbi Decoder. single_bit_error usually simulates one encoded bit being flipped; burst_error_short simulates a short segment of continuous errors. They are not intended to give real channel statistical results, but to verify whether the decoder, when facing local errors, corrects errors through the accumulated cost of the entire trellis path. If the decoder only performed bit-by-bit hard decisions, it could only see the current received bits and would more easily be misled by local errors; while Viterbi Decoder compares the path metrics of complete candidate paths, so when the number of errors does not exceed the correction capability, it may still recover the correct payload.

- AWGN soft cases：soft_awgn_0db, soft_awgn_1db, soft_awgn_2db are used to check whether soft-decision input can work normally in a noisy environment. AWGN is Additive White Gaussian Noise, used to simulate random noise superimposed during signal transmission. The lower the SNR, the stronger the noise relative to the signal, so 0 dB is more difficult than 1 dB and 2 dB; 2 dB is relatively easier. The focus of soft-decision cases is not only to look at the final 0/1, but to check whether 3-bit soft symbol, soft branch metric, finite path metric width, and subtract-min normalization cooperate correctly under noisy input. The parameter sweep stage found that when traceback depth is only 16, soft_awgn_0db produces mismatches; when depth increases to 32, 40, 64, mismatch becomes 0 under the current vector set. This indicates that under low SNR conditions, the path needs sufficient traceback depth to converge stably, and making decisions too early may cause an incorrect path to be selected.

The approximate limited-vector BER results are as follows：

Table 4：Limited-vector mismatch results.

Analysis object	Data source	Observed result	Explanation
hard-decision RTL simulation	`data/regression/rtl_regression_summary.csv`	All 4 cases 0 mismatch	Basic hard baseline correct
soft-decision RTL simulation	`data/regression/soft3_regression_summary.csv`	All 3 AWGN cases 0 mismatch	soft3 hero correct under selected parameters
traceback depth sweep	`data/analysis/traceback_depth_sweep.csv`	depth=16 has 8 mismatches, 32/40/64 are 0	Too short traceback loses path convergence
path metric width sweep	`data/analysis/path_metric_width_sweep.csv`	Under depth=40, 8/10/12/16 bit are all 0 mismatch	The current vector set did not expose insufficient width
board UART run	`data/board_runs/board_summary.csv`	Final run all 3 cases 0 mismatch	Real board-level data path passed

It needs to be especially emphasized that the path metric width sweep results looking “all 0” does not mean that bit width is never important. Path metric width determines how many bits can be used in hardware to store the accumulated path cost. If the frame is longer, the branch metric is accumulated more times; if the noise is stronger, the metric distribution between different paths may be more complex; if subtract-min normalization is turned off, the absolute value of path metric will continue to grow. Under these circumstances, a smaller bit width may overflow or truncate, thereby destroying the relative magnitude between paths and ultimately affecting the

ACS selection. Therefore, the current result can only say: under 96-bit payload, current AWGN vectors, current subtract-min normalization, and current test scale, no mismatch caused by insufficient bit width was observed. If later a BER conclusion closer to the meaning of a communication paper is needed, longer random frames, more SNR points, and larger sample counts need to be extended.

10. Synthesis and Implementation Results

Vivado results need to be interpreted in layers [4]. Passing synthesis only means that RTL can be converted by the tool into FPGA resources such as LUT, FF, BRAM, and DSP; passing implementation only means placement and routing have been completed, that is, resources have been placed at specific positions on the chip and wiring has been completed; but whether the design can operate stably at the target frequency still depends on WNS and TNS in the timing report. Therefore, here we cannot only look at the return code of Vivado commands, but must also look at resources, timing, and power.

Table 5：Vivado synthesis and implementation results.

Design version	Stage	LUT	FF	BRAM	DSP	WNS(ns)	TNS(ns)	Estimated power(W)
hard-decision core	synthesis	8116	15243	0	0	6.956	0.000	0.410
soft-decision core	synthesis	10570	17118	0	0	-23.241	-17607.301	0.469
soft-decision core	implementation	10647	17118	0	0	-20.433	-16272.939	0.487

The data source of this table is data/impl/vivado_summary.csv. LUT and FF reflect logic and register resource usage, BRAM and DSP reflect whether on-chip block RAM and dedicated arithmetic hard blocks are used, WNS and TNS reflect whether timing satisfies the target constraint, and estimated power comes from Vivado’s power report. The WNS of the hard-decision core is positive, indicating that it still has timing margin under the 10 ns clock constraint; the WNS of the soft-decision core is negative, indicating that although it can complete synthesis and placement/routing, it cannot operate stably under the 100 MHz target frequency.

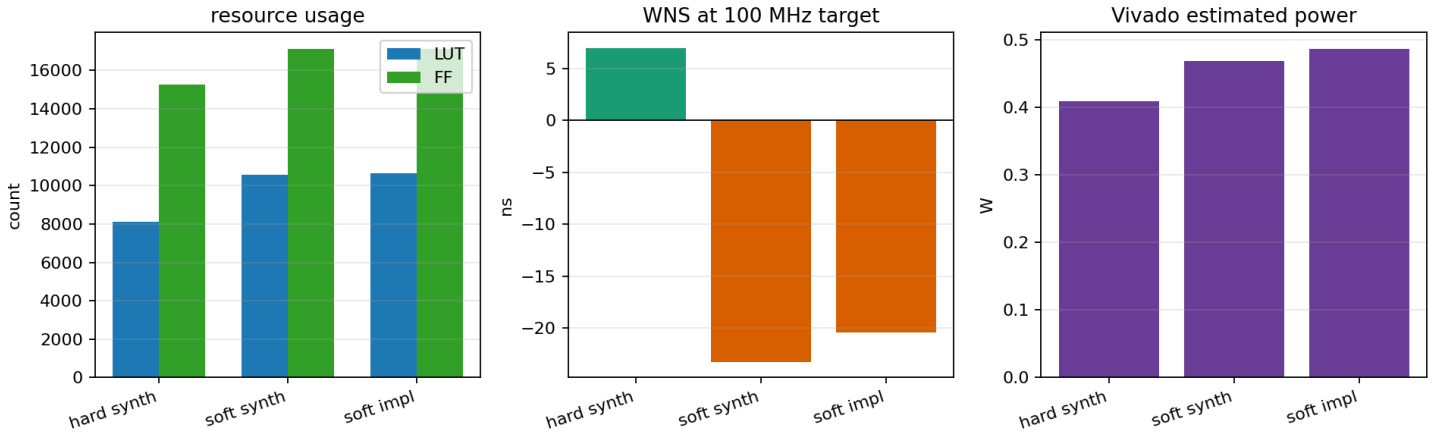

Figure 8：Vivado resource timing power results. This figure shows a comparison of Vivado resources, timing, and power. The left figure compares LUT and FF, showing that the soft-decision core indeed increases resources compared with the hard-decision core, but the increase remains within an explainable range; the middle figure compares WNS, showing that the timing of the soft-decision core deteriorates significantly; the right figure compares Vivado estimated power, showing that the power estimate of the soft-decision core also increases. After looking at resources, timing, and power separately, it can be found that the main problem of the current project is not that FPGA resources are completely insufficient, but that the critical path of the soft-decision version is too long, causing 100 MHz timing closure failure.

The results of the hard-decision core show that the basic Viterbi architecture can be processed relatively smoothly by the tool. Its LUT is 8116, FF is 15243, and WNS is 6.956 ns. 100 MHz corresponds to a 10 ns clock period, and positive WNS indicates that the slowest path is still faster than the clock requirement, meaning the hard-decision version has large timing margin under this constraint. This result also shows that the basic structures such as trellis update, path metric storage, and traceback themselves are not impossible to implement. The real pressure mainly comes from the more complex metric computation and normalization logic in the soft-decision version.

The synthesis result of the soft-decision core shows that LUT increases to 10570 and FF increases to 17118. This is expected, because soft-decision no longer only compares whether 0/1 are the same, but needs to process 3-bit soft symbols. BMU needs to map expected 0 to 0 and expected 1 to 7, and calculate the absolute distance from the received symbol to the ideal value; path metric also needs to preserve finer accumulated cost; subtract-min normalization also needs to find the minimum value among multiple states' path metrics and subtract it uniformly. Therefore, the soft-decision version introduces more combinational logic, registers, and comparison paths.

The implementation result of the soft-decision core is the most important negative result of this project. Although placement and routing completed, WNS is -20.433 ns, and TNS is -16272.939 ns. Negative WNS indicates that the worst path did not complete within the target clock period; TNS is negative and has a large magnitude, indicating that there is more than one failing path and the overall timing pressure is relatively obvious. The critical path pointed to by the Vivado timing report goes from compute_idx_reg[4]/C to metrics_reg[47][10]/D, with data path delay of 30.415 ns, including logic delay of 13.839 ns, routing delay of 16.576 ns, and logic levels reaching 116 layers. This result indicates that the current design undertakes too much logic within one clock

cycle. Especially after 64-state soft ACS update and subtract-min reduction are combined, an overly long combinational path is formed.

Therefore, the soft-decision hero design is functionally correct and can be synthesized and implemented by Vivado, but it is currently not a 100 MHz timing-clean design. The focus of later optimization should be placed on splitting critical paths, for example adding pipeline in the ACS update path, splitting addition, comparison, selection, and normalization into multiple cycles; performing more balanced structural optimization on the reduction tree; or adjusting register positions through retiming to reduce the number of combinational logic layers crossed within a single clock cycle.

11. Timing, Throughput, Latency, and Power Analysis

Timing analysis first needs to look at the target frequency. This project originally hoped that the soft-decision core could run at 100 MHz, and 100 MHz corresponds to a clock period of $T = 1/f = 1/100\ \mathrm{MHz} = 10\ \mathrm{ns}$. That is, data output from one register must pass through intermediate combinational logic and arrive at the next register within 10 ns while remaining stable. If the delay of a combinational path exceeds 10 ns, the data sampled by the target register at the next clock may not yet be stable, and the design will experience timing fail.

WNS in the Vivado timing report is used to describe how much time the worst path still has before satisfying the clock constraint. A positive WNS means the worst path still satisfies timing; a negative WNS means the worst path exceeds the clock period requirement. Therefore, the positive WNS of the hard-decision core indicates that it satisfies the 10 ns constraint; the negative WNS of the soft-decision core indicates that although it is functionally correct, it cannot be directly used as a timing-clean design at 100 MHz.

Throughput needs to distinguish theoretical perspective and board-level measured perspective. Theoretically, if the decoder core can continuously output 1 decoded bit per cycle in an ideal steady-state state, then at 100 MHz the maximum throughput can approach 100 Mbit/s. But the board-level test of this project does not only measure the pure decoder kernel’s steady-state output. Instead, it records the total cycles of an entire frame from start to completion. This total cycles includes control register start, DMA data movement, core processing, completion waiting, result readback, and UART verification-related overhead. Therefore, the report cannot directly use the theoretical 1 bit per cycle to claim actual throughput, but should conservatively estimate based on the whole-frame cycles recorded in the UART log.

The final board-level run records are as follows：

Table 6：Board-level whole-frame throughput estimate.

case	decoded bits	cycles	frame time at 25 MHz(us)	throughput at 25 MHz(Mbit/s)	estimate at 100 MHz with same cycles(Mbit/s)
no_noise	96	802	32.08	2.99	11.97
single_bit_error	96	855	34.20	2.81	11.23

case	decoded bits	cycles	frame time at 25 MHz(us)	throughput at 25 MHz(Mbit/s)	estimate at 100 MHz with same cycles(Mbit/s)
soft_awgn_2db	96	855	34.20	2.81	11.23

The 25 MHz frame time in this table is obtained from $t_{\text{frame}} = \text{cycles}/f_{\text{clk}}$. For example, no_noise case uses 802 cycles. At 25 MHz, each cycle is 40 ns, so the whole-frame time is $802 \times 40\ \text{ns} = 32.08\ \text{us}$.

Throughput is obtained from $R = \text{decoded bits}/t_{\text{frame}}$, so the whole-frame throughput of the no_noise case is approximately $96/32.08\ \text{us} = 2.99\ \text{Mbit/s}$. The other two cases use 855 cycles, corresponding to a frame time of 34.20 us and a whole-frame throughput of about 2.81 Mbit/s.

The last column's 100 MHz same-cycle estimate only puts the same cycles under a 100 MHz clock and recomputes it, used to observe the throughput order of magnitude if the board-level flow cycle count remains unchanged and the clock is increased to 100 MHz; it is not an actual 100 MHz board-level passing result, because the current soft-decision design has not completed timing closure at 100 MHz

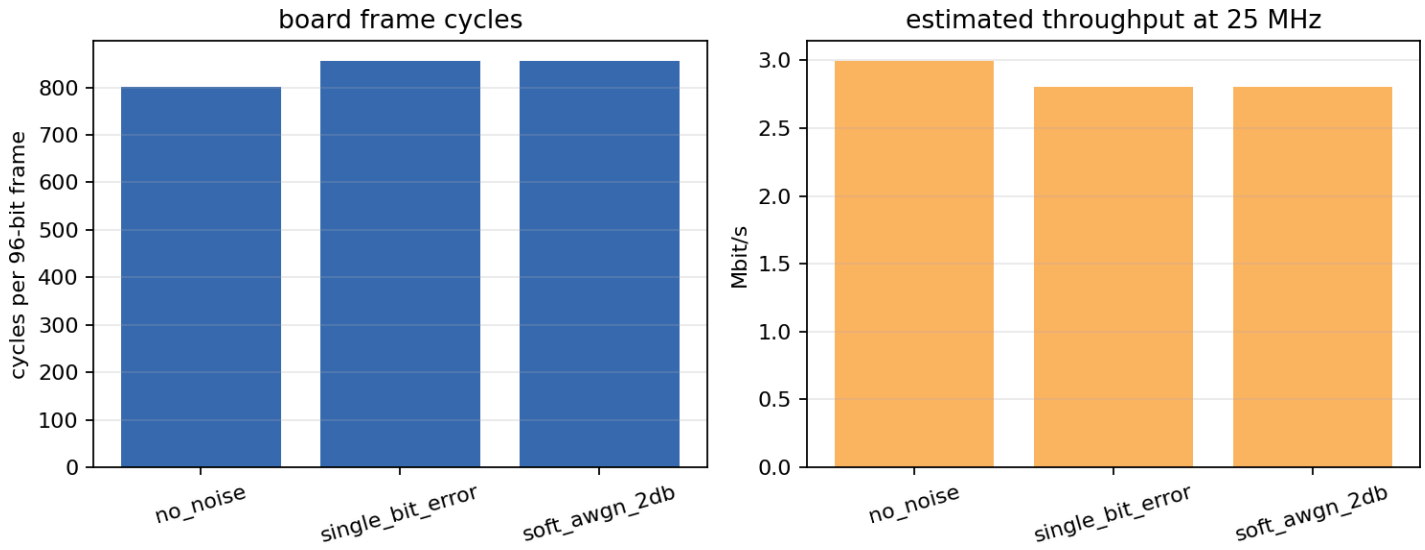

Figure 9：Board-level cycles and whole-frame throughput. The left figure shows the completion cycles of each 96-bit frame, and the right figure converts it into throughput under the 25 MHz board shell. no_noise has fewer cycles, while the two cases with errors or noise both have 855 cycles.

Latency also needs to be interpreted carefully. Strictly speaking, latency can have multiple definitions, such as first-valid latency from the first input symbol to the first output valid bit, or frame latency from starting one frame decode to completing the entire frame. This UART log records the whole-frame completion cycles, not the first-valid output cycle. Therefore, the report cannot fabricate first-valid latency, and can only state that in the final board run, no_noise used 802 cycles to completely process one frame, and single_bit_error and soft_awgn_2db used 855 cycles to completely process one frame. If first-valid latency needs to be accurately reported in the future, an additional cycle record of when the first output valid appears needs to be added in RTL or control registers, for example adding a first_valid_cycle counter and printing it in the UART log.

Power also needs to distinguish sources. The 0.487 W given by Vivado is a vector-less power estimate of the soft-decision implementation under 100 MHz conditions. Here vector-less means that the tool did not use full switching activity data from real board operation, but estimated power based on design structure and default activity rates. Therefore, it can serve as a reference for the order of magnitude of design power, but it is not real power measured by board power instruments. On the other hand, the final board shell completed operation at 25 MHz, and the clock frequency is also different, so 0.487 W cannot be directly treated as the actual 25 MHz board-level power consumption.

If only an order-of-magnitude estimate under the same conditions is made, Vivado's 0.487 W and the 100 MHz same-cycle throughput can be used to calculate energy per bit：

Table 7：Energy order-of-magnitude estimate.

case	100 MHz same-cycle throughput(Mbit/s)	energy estimated using 0.487 W(nJ/bit)
no_noise	11.97	40.7
single_bit_error	11.23	43.4
soft_awgn_2db	11.23	43.4

These energy numbers are calculated by $E_b = P/R$. For example, the 100 MHz same-cycle throughput of the single_bit_error case is about 11.23 Mbit/s. Using 0.487 W for estimation gives an energy of about 43.4 nJ/bit. This result can only show the approximate energy order of magnitude under Vivado estimated power and whole-frame throughput perspective, and cannot be used as a real board-level energy conclusion. A truly rigorous power evaluation requires board-level power measurement, or at least power analysis with real switching activity, for example generating SAIF/VCD from simulation and then importing it into Vivado for power analysis.

12. Board-Level System Design and Hardware Test

The role of the board-level system is to connect the decoder core that has already passed RTL simulation to the real ZU4EV development board for operation. At this step, the verification object is no longer just the standalone Viterbi core, but the entire PS/PL system.

The software part runs on the ARM processor of PS（Processing System）, responsible for preparing input data, configuring DMA, starting the hardware decoder, reading output results, and comparing with golden output [5]；

The hardware Viterbi decoder runs in PL（Programmable Logic）, responsible for actually executing the decoding computation.

Input and output data are moved between PS and PL through AXI DMA, UART is used to print test results back to the computer, and JTAG is used to download bitstream and ELF.

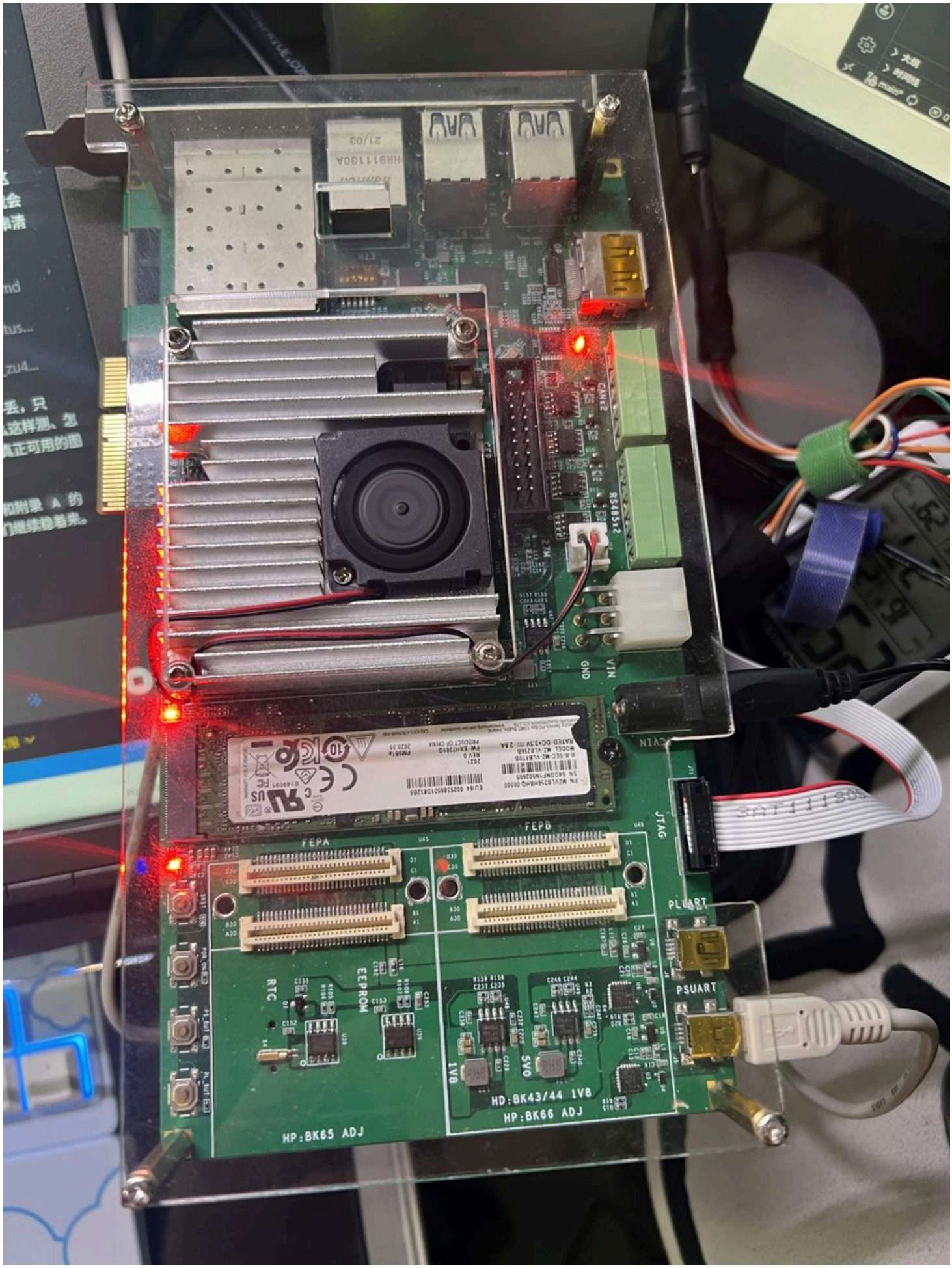
FEPA
FEPB
JTAG
PLUART
PSUART
RTC
EEPROM
1V8
5V0
HD:BK43/44 1V8
HP:BK66 ADJ
HP:BK65 ADJ
GND
VIN
RS485x2

Figure 10：ZU4EV development board physical photo. This figure is used to explain that this project does not only stay at simulation, but is connected to a real development board. During board-level verification, the computer downloads hardware configuration and bare-metal program through JTAG, and captures UART log through COM9 serial port.

The board-level data flow from top to bottom is as follows：

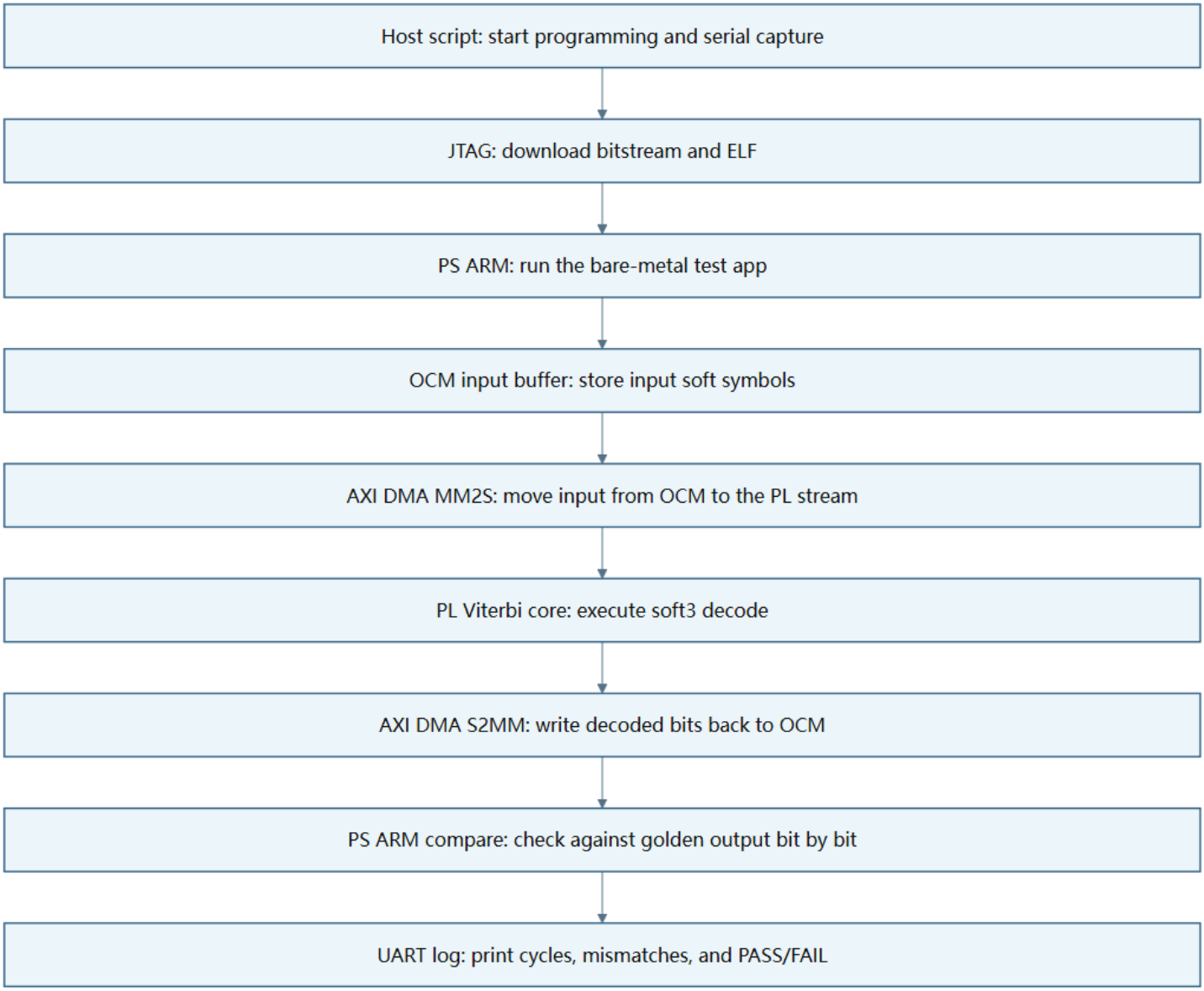

Figure 11：Board-level data flow. This figure shows the complete data path of one board validation. First, the computer-side script starts the test flow and completes bitstream and ELF download through JTAG. Then, the bare-metal program running on ARM writes test input into the OCM buffer and configures AXI DMA. The MM2S channel of DMA sends input soft symbols from memory into the PL-side Viterbi core; after the core completes decoding, it outputs decoded bits through AXI Stream; the S2MM channel of DMA writes decoded bits back to OCM. Finally, the ARM program reads the output buffer, compares it bit by bit with golden output, and prints each case's PASS/FAIL, mismatch count, and cycle count through UART. In other words, board-level verification checks the complete data link, not just the local function of decoder core.

There is an important debugging point here: where to place the buffer. In the first attempt using DDR buffer, the decoder core did not see valid stream input, indicating that although data was prepared on the software side, it was not truly sent to PL through DMA. After switching to OCM buffer later, the core could see input, indicating that the data path had been partially connected, but a DMA decode error then occurred. Continuing to compare the Vivado address map, the problem was located to the OCM segment not being included in the DMA's MM2S/S2MM accessible address space. In other words, the address range accessed by DMA did not match the address segment where the actual buffer was located. After adding the OCM segment address mapping, DMA could correctly access the OCM buffer, and the final board run passed.

The CMD log screenshot is as follows.

```
Command: Get-Content data\board_runs\m7_board_20260502_083547\uart.log
UART success log excerpt
FSBL is running on RPU.
Warning: APU-only restart is not supported if FSBL boots on RPU.
ZU4EV Viterbi bare-metal harness
stage=platform_init_begin
DMA buffers tx=0x00000000FFFCFD00 rx=0x00000000FFFCFEC0
stage=platform_init_done
stage=dma_init_begin
stage=dma_init_done
Console=COM9, arch_id=7
stage=no_noise:case_begin input_len=204 output_len=96 run_id=m2_no_noise
[no_noise] dma_state@after_start mm2s_cr=0x00010003 mm2s_sr=0x00000000 s2mm_cr=0x00010003 s2mm_sr=0x00000000
[no_noise] shell_state@after_start status=0x00000001 cycles=802 seen=204 emitted=96 svalid=204 sready=500
[no_noise] len=96 cycles=802 mismatches=0 status=0x00000001
stage=single_bit_error:case_begin input_len=204 output_len=96 run_id=m2_single_bit_error
[single_bit_error] dma_state@after_start mm2s_cr=0x00010003 mm2s_sr=0x00001000 s2mm_cr=0x00010003 s2mm_sr=0x00001000
[single_bit_error] shell_state@after_start status=0x00000001 cycles=855 seen=204 emitted=96 svalid=204 sready=553
[single_bit_error] len=96 cycles=855 mismatches=0 status=0x00000001
stage=soft_awgn_2db:case_begin input_len=204 output_len=96 run_id=m2_soft_awgn_2db
[soft_awgn_2db] dma_state@after_start mm2s_cr=0x00010003 mm2s_sr=0x00001000 s2mm_cr=0x00010003 s2mm_sr=0x00001000
[soft_awgn_2db] shell_state@after_start status=0x00000001 cycles=855 seen=204 emitted=96 svalid=204 sready=553
[soft_awgn_2db] len=96 cycles=855 mismatches=0 status=0x00000001
Completed 3 cases, failures=0
```

Figure 12： Final UART log screenshot. The figure only keeps relative paths and UART output excerpts, and does not include local absolute paths.

This screenshot shows the real output of the final ZU4EV board validation.

At the beginning of the log, it can be seen that the bare-metal harness has started and completed platform_init and dma_init; then it sequentially runs three cases: no_noise, single_bit_error, and soft_awgn_2db. For each case, input_len=204 and output_len=96, indicating that the ARM side sent 204 input samples to PL and expected the decoder to output 96 decoded bits.

During operation, seen=204 indicates that the PL side indeed received the complete input, and emitted=96 indicates that the decoder output the complete 96-bit result. The final line of all three cases shows mismatches=0, and the final summary is Completed 3 cases, failures=0, indicating that the real board-level data path has run through, and board output is completely consistent with golden output.

The final case results are as follows.

Table 8：Final board-level case results.

case	input samples seen	output samples emitted	decoded bits	cycles	mismatches	result
no_noise	204	96	96	802	0	PASS
single_bit_error	204	96	96	855	0	PASS
soft_awgn_2db	204	96	96	855	0	PASS

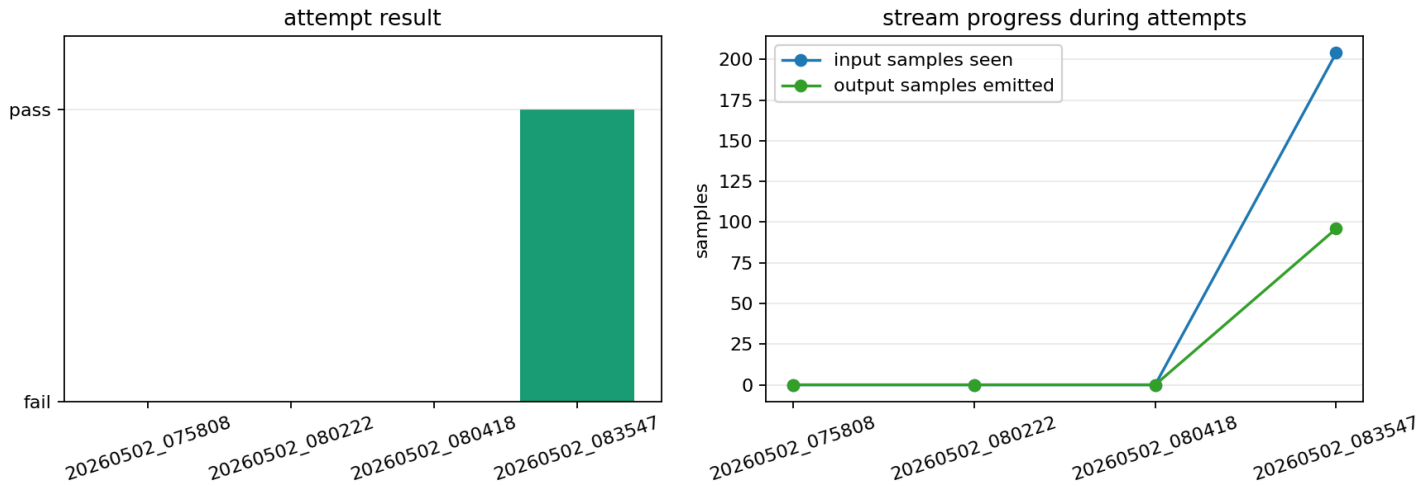

Figure 13：Board-level attempt process. This figure does not only look at the final success, but also shows the previous failures. The left side shows whether each attempt passed or failed; the right side shows how many input samples the stream saw and how many output samples it emitted. The first timeout means the serial port did not wait for the completion flag, indicating that the system did not fully finish running. In the second functional failure, the core had already seen 109 samples, but emitted was still 0. This indicates that the problem was not that the app completely failed to start, but that DMA/address mapping or stream transfer had an error midway. In the final pass, seen=204 and emitted=96, indicating that the input fully entered PL, and the output also fully returned to PS.

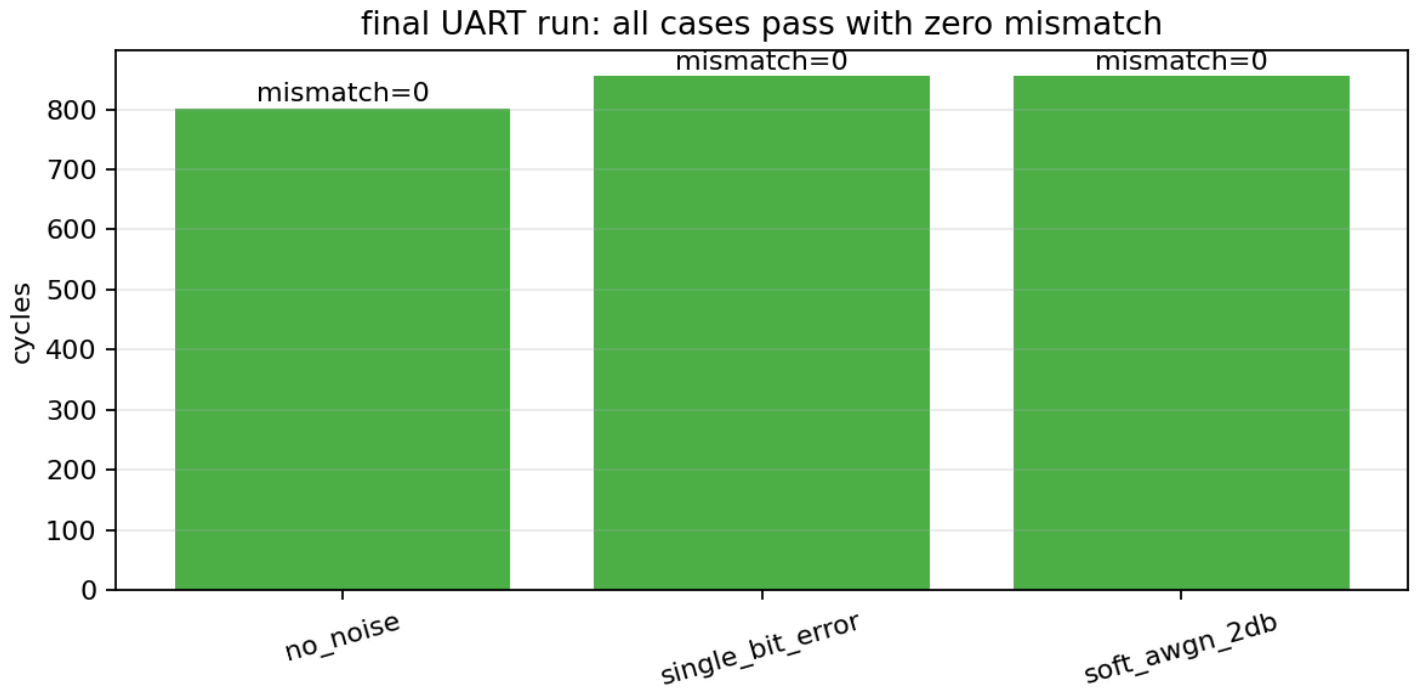

Figure 14：Final UART case results. All three cases have mismatch=0. The cycles values also correspond to the results printed in the UART log.

13. Bottleneck Analysis and Architectural Tradeoffs

This project is not intended to reinvent the Viterbi algorithm. Viterbi Decoder itself is a classical decoding algorithm, and rate-1/2, K=7, [171,133] is also a common convolutional code configuration. The focus of this project is to truly place this algorithm into an FPGA engineering closed loop, and clearly explain the key tradeoffs in the implementation process: why soft-decision can provide more reliability information, why traceback depth cannot be too short, why path metric width cannot be increased indefinitely, why timing may still fail after functional correctness, and why the real board shell will also expose DMA, cache, and address mapping problems.

The first bottleneck is the recursive dependency of Viterbi itself. Viterbi's path metric is accumulated step by step, and the path metric of the current trellis step must be derived from the path metric of the previous step. Therefore, it cannot be arbitrarily parallel-expanded like completely independent combinational computation. Each step must wait for the previous round of state update to complete before continuing to calculate the next round of states. This dependency limits the space for throughput improvement, and also makes pipeline design require special care, because the sequence between trellis steps must not be destroyed.

The second bottleneck is 64-state ACS update. Since the formal configuration is K=7, there are $2^{K-1} = 64$ states in the trellis, and each state needs to select the smaller path metric from candidate predecessor paths at every step. The branch metric of the hard-decision version is only simple Hamming distance, so the logic is relatively light; the soft-decision version needs to process 3-bit soft symbols, calculate the distance to ideal 0 or 7, and add these distances to path metric. In the current implementation, a lot of logic such as 64-state update,

soft metric calculation, comparison, and selection is concentrated into one cycle, so a very long critical path appears in the Vivado timing report, and the report shows that the critical path reaches 116 logic levels.

The third bottleneck is subtract-min normalization. Its role is to control the numerical range of path metrics and avoid the fixed-width register overflow caused by accumulated cost continuously growing in long frames. Its advantage is numerical stability, and its disadvantage is nontrivial hardware cost: every trellis step needs to first find the minimum value among 64 path metrics, and then make all states' path metrics simultaneously subtract this minimum value. This minimum reduction and subsequent subtraction increase combinational logic depth and routing pressure. Therefore, normalization has value in function and numerical stability, but it also increases the difficulty of timing closure.

The fourth bottleneck is survivor memory and traceback. Viterbi Decoder does not save every complete path, but saves the survivor decision of each state at each time step, and finally recovers the input sequence through traceback. The larger the traceback depth, the more stable the path usually is, and the less likely the decoding result is affected by short-term noise; but increasing depth brings more survivor storage, longer waiting time, and more complex read/write control. Parameter sweep shows that depth=16 produces mismatch under the current soft_awgn_0db vector, while depth=32, 40, 64 all pass under the current vector set. Therefore, depth=40 is finally selected as a compromise among path stability, storage overhead, and latency.

The fifth bottleneck is the board shell. In the simulation environment, the testbench can directly send input to RTL and directly check output; but on the real development board, data must pass through a complete link including PS, DMA, OCM or DDR buffer, AXI Stream, TLAST, valid/ready, cache flush/invalidate, and UART log. Any error in any link will manifest as the core not receiving data, abnormal DMA status, incorrect output length, or mismatch. The final debugging process shows that a functionally correct decoder core is only a necessary condition for successful board bring-up, not a sufficient condition; only when system integration is also correct can the design truly run through.

Therefore, the significance of this project does not lie in proving the known conclusion that "Viterbi can decode", but in completing a reproducible engineering verification chain: from Python golden model, to RTL bit-true simulation, to parameter sweep, then to Vivado resource and timing analysis, and finally to real ZU4EV UART PASS. At the same time, this project also preserves an important negative result: the soft-decision hero design currently has not reached 100 MHz timing closure. This result indicates that the current architecture has already closed the loop functionally, but still needs pipeline, retiming, or ACS/reduction structure optimization for high-frequency implementation

14. Conclusion and Future Work

This project has already completed a submit-ready end-to-end version

- At the algorithm level, the Python model provides a golden reference
- At the test level, unified test vectors let the software model, RTL simulation, and real board-level verification use the same batch of inputs and expected outputs

- At the RTL level, both the hard-decision baseline and the soft-decision hero design passed bit-true regression
- At the parameter selection level, the choices of traceback depth and path metric width are supported by sweep results
- At the tool level, Vivado synthesis, implementation, resources, timing, and power results have been recorded
- At the board level, the latest ZU4EV UART run shows all 3 cases 0 mismatch

Therefore, the current version is no longer a single-point functional demonstration, but has completed a full verification closed loop from algorithm to hardware, and from simulation to board

The clearest conclusion at present is that the functional closed loop has passed, but high-frequency timing has not been completed. Under the 100 MHz target, the soft-decision hero has WNS of -20.433 ns, which indicates that the 64-state soft ACS, normalization, and metric update completed within one clock cycle are too heavy. Board-level verification passed at 25 MHz, meaning that the current version is a functionally correct, reproducible implementation version that can run on board. Later optimization should shift focus to timing closure and long-frame performance evaluation

Future work mainly includes the following directions

- Pipeline：The most direct direction is to insert pipeline stages between the ACS array and subtract-min normalization, splitting the addition, comparison, minimum search, and subtraction originally completed in one clock into multiple clocks. This will increase latency, but has the opportunity to significantly improve WNS.
- Normalization reorganization：Current subtract-min performs a global minimum search at every step, which creates high timing pressure. Hierarchical reduction, normalizing once every several steps, or using saturation strategies for comparison can be considered. However, all of these require re-verifying fixed-point behavior, and RTL cannot be changed without changing the golden model.
- Real BER campaign：The current report only uses limited 96-bit vectors, and cannot claim a complete BER curve. Later, longer random payloads can be generated for multiple SNR points, accumulating enough bits at each point, then plotting BER vs SNR curves. Only in this way can the statistical performance improvement of soft-decision relative to hard-decision be evaluated.
- DDR DMA：The current final passing version uses OCM buffer. Its advantage is that the address path is simple and the capacity is sufficient for the current cases; its disadvantage is limited capacity. If longer frames or batch BER are to be run later, DDR buffer needs to be made functional, and cache flush/invalidate and address mapping must be strictly handled.
- Survivor memory optimization：The current design leans more toward functional closed loop and debuggability. Later, survivor storage can be more systematically mapped to BRAM or a more compact RAM structure, reducing FF pressure and making traceback more suitable for long frames.

Due to limited time, this project prioritized completing a reproducible end-to-end functional closed loop and real ZU4EV board-level verification. If there is an opportunity later, work will mainly focus on 100 MHz timing closure, long-frame BER testing, and board-level data path optimization. These are not implemented for now

15. AI Tools Statement

AI tools were used as engineering assistants for planning, code organization, script generation, debugging guidance, and report drafting. Final architectural decisions, validation criteria, experiment interpretation, and submission responsibility remain with the author.

16. References

[1] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," IEEE Transactions on Information Theory, vol. 13, no. 2, pp. 260-269, Apr. 1967, doi: 10.1109/TIT.1967.1054010.

[2] G. D. Forney, Jr., "The Viterbi algorithm," Proceedings of the IEEE, vol. 61, no. 3, pp. 268-278, Mar. 1973, doi: 10.1109/PROC.1973.9030.

[3] S. Lin and D. J. Costello, Error Control Coding, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2004.

[4] AMD, Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906). AMD Technical Information Portal. [Online]. Available: https://docs.amd.com/r/2022.2-English/ug906-vivado-design-analysis

[5] AMD, Zynq UltraScale+ Device Technical Reference Manual (UG1085). AMD Technical Information Portal. [Online]. Available: https://docs.amd.com/v/u/en-US/ug1085-zynq-ultrascale-trm